FeatureSAGE: Stable Attribute Group Editing With Integrated StyleFeatureEditing Encoder for Enhanced Few-shot Image Generation

A Thesis Proposal by

Joshua Libando

Sam Joash V. Antonio

Submitted to the Department of Computer Science

College of Computing and Information Science (CCIS)

Caraga State University – Main Campus

In Partial Fulfillment

of the Requirements for the Degree

Bachelor of Science in Computer Science (BSCS)

February 10, 2026

CONTENTS

LIST OF FIGURES

ABSTRACT

CHAPTER 1 INTRODUCTION

1.1 Background of the Study

Generative Adversarial Networks (GANs) have become a foundational framework for high-fidelity image generation. In a GAN, a generator and a discriminator are trained in a minimax game: the generator learns to produce synthetic images that fool the discriminator into classifying them as real, while the discriminator learns to distinguish generated images from real ones (Goodfellow et al., 2014). This adversarial process has enabled GANs to achieve remarkable realism in generated images (e.g. faces, animals) across many domains. In particular, the StyleGAN family of architectures (Karras et al., 2020) introduced a novel style-based generator that has set new standards for image quality. StyleGAN maps an input latent code z through a learned mapping network into an intermediate latent w in a space called W. Each layer of the generator is then modulated by w via adaptive instance normalization (AdaIN), and stochastic noise inputs add fine detail. This style-based design yields an intermediate latent space W that is significantly more disentangled than the original input space Z. As a result, controlled manipulation of individual semantic attributes (e.g. hair color, pose) is possible by adjusting w or the per-layer style codes. Subsequent improvements (StyleGAN2, StyleGAN3) refined the architecture and training, further boosting image fidelity and easing inversion (mapping real images back to latent codes).

StyleGAN's latent representations come in several forms. The basic mapping from Z to W produces a single 512-D style code w per image. An extended latent space W^+ is also used, where a separate w vector is specified for each of the 18 synthesis layers. An even higher-dimensional feature space F can be formed by concatenating intermediate style and feature tensors (sometimes denoted F_k), capturing spatial details in the

generator. Recent work has highlighted a trade-off across these spaces: moving from W to W^+ to F increases reconstruction fidelity but reduces editability (Liu et al., 2023) . In other words, encoding an image in the low-dimensional W space yields highly edit-friendly representations but may lose fine detail, whereas the high-dimensional W^+ or F spaces can reconstruct details at the cost of some entanglement and overfitting. StyleGAN2 in particular has shown that inversion in W^+ benefits from regularization (e.g. path-length regularization) to improve reconstructability and make projected images easier to attribute to latent codes (Karras et al., 2020).

To edit real images with a GAN, one must first perform GAN inversion: find a latent code that generates (nearly) the given image. This is a well-studied but challenging problem (Bobkov et al., 2024; Richardson et al., 2021). Early inversion methods used optimization (iteratively updating z to minimize reconstruction error), which can achieve high fidelity but is slow and may produce out-of-distribution latents. Encoder-based methods train a neural network to predict the latent directly, enabling real-time inversion at the expense of some quality. For StyleGAN, Richardson et al. (2021), proposed Pixel2Style2Pixel (pSp): a feed-forward encoder that maps any input image to a sequence of style vectors in W^+ (one w per layer). The pSp encoder can invert faces (or other images) into W^+ accurately without test-time optimization, greatly simplifying downstream editing. Generally, as noted by Richardson et al. (2021), "inversion methods either directly optimize the latent vector to minimize the error for the given image". Encoder approaches like pSp offer fast inference, while optimization-based methods remain stronger on pure reconstruction quality. In either case, the goal of GAN inversion is to find an internal representation that contains as much image detail as possible and still allows semantic edits (Bobkov et al., 2024).

Once a real image is embedded in StyleGAN's latent space, attribute editing becomes possible. It has been observed that semantic attributes (e.g. "smile," "age," "eyeglasses") correspond to linear directions or subspaces in the latent space. For

example, Shen et al. (2020) showed that well-trained StyleGAN latent codes encode a disentangled representation under linear transformations. In practice, one can move the latent w of an image along a learned direction Δw to change a particular attribute (e.g. $w + \alpha\Delta w$) (Ding et al., 2022). Many works have exploited this: e.g. Linear SVM or PCA over latent codes can find directions for "smile" vs "no smile," or for pose changes This allows controlled editing via vector arithmetic. Importantly, style-based latents like W and W^+ are far less entangled than raw Z, so edits tend to be more semantically meaningful (Karras et al., 2020; Shen et al., 2020). Still, some entanglement remains changing one attribute may inadvertently affect others. Recent methods (e.g. using subspace projection or network-based attribute classifiers) seek to further disentangle attributes. Moreover, higher-dimensional latent spaces (like W^+ or F) can encode finer identity details, enabling extremely high-quality reconstructions. Bobkov et al. (2024) leverage this by introducing a StyleFeatureEditor: an encoder that predicts both a W^+ code w and an intermediate feature tensor F_k. By editing in both w and F_k, they reconstruct and preserve finer image details even under heavy edits. In summary, StyleGAN latent spaces provide a powerful substrate for attribute editing, but the choice of W vs W^+ vs F presents a trade-off between edit stability and detail, and sophisticated encoders are needed to balance both.

A parallel line of research addresses few-shot image generation and editing. Unlike standard GAN training (which requires large datasets), few-shot generation aims to adapt a model to a new category or attributes given only a handful of examples. Meta-learning approaches have been applied here: for instance, the FIGR model Clouâtre and Demers (2019) meta-trains a GAN with the Reptile algorithm so it can generate new classes from as few as 4 examples. Other works like FUNIT (Liu et al., 2019b) tackle few-shot domain translation, using image encoders to condition style generation on a few references. However, generating realistic variation from very limited data remains challenging. In particular, attribute group editing has emerged as a promising strategy

for few-shot GANs. The idea is to collect many examples of known categories to learn generic attribute directions, then apply those directions to few-shot target categories. AGE learns a dictionary of "category-irrelevant" attribute vectors (e.g. pose, color) from seen classes and "category-relevant" class embeddings (mean latent per class) (Ding et al., 2022). At inference, given one or a few examples of an unseen category, AGE embeds them into W^+ (using pSp) and manipulates those latents by combining the learned attribute directions. Similarly,Zhang et al. (2024) propose a Transformer-style approach with modules for learning discrete attribute codebooks and prompt-driven editing to handle diverse unseen classes. These methods show that by factorizing and reusing attribute vectors, a pre-trained GAN can generate plausible images of new categories without full retraining (Ding et al., 2022). Nonetheless, they often assume large amounts of labeled data for the seen categories and focus on one attribute at a time or on broad class attributes (like "breed of dog" vs "background color of bird"), rather than coordinated multi-attribute control.

Despite these advances, important gaps remain. Most GAN inversion and editing methods have focused on single-attribute manipulations or on scenarios with abundant data. Achieving stable, coordinated multi-attribute edits is still difficult, especially under few-shot conditions. For example, existing StyleGAN inversion methods either excel at reconstruction but produce entangled edits, or vice versa (Liu et al., 2023; Richardson et al., 2021). Moreover, according to Ding et al. (2023) , "class-inconsistency" is a common-problem GAN-generated images for downstream classification. Class inconsistency happens when the generative model corrupts some minor but critical information versus real pictures of a certain category (Ding et al., 2023). Few-shot models like AGE and TAGE demonstrate that group editing is possible, but they often do not explicitly ensure fidelity or identity preservation when combining many edits. Moreover, generation quality in few-shot setups can degrade due to mode collapse or identity drift. In face editing, for instance, we desire that an edited

face remains clearly the same person (high identity similarity) while attributes like expression or hair style change. Conventional metrics like Fréchet Inception Distance (FID) and LPIPS capture distribution quality and perceptual similarity (Karras et al., 2020; Zhang et al., 2018b).

We proposed that achieving improved and stable group-wise attribute editing under few-shot conditions required an integrated approach. FeatureSAGE is designed to address this by combining replacing the standard pSp encoder of SAGE with StyleGANEditor Inverter and Feature Editor. The integrated encoder (inspired by StyleFeatureEditor) jointly predicts a W^+ latent w and a feature tensor F, aiming for high-fidelity inversion as well as explicit control of feature-level detail. This architecture is intended to preserve identity and image realism (as measured by identity similarity and low FID/LPIPS) while achieving disentangled, controlled modifications of target attributes. In summary, the background works on GANs, StyleGAN latent editing, and few-shot group editing together motivate FeatureSAGE's goal: to provide stable, coordinated multi-attribute control in the latent space of a pre-trained GAN, even with only limited example images, without sacrificing image quality or attribute disentanglement.

1.2 Statement of the Problem

This study examines the main limitations of the current SAGE approach, specifically the use of the Pixel2Style2Pixel (pSp) encoder, which tends to yield latently entangled codes and diminished control over semantic edits. The limitations reduce the creation of semantic-consistent, high-fidelity images from few-shot instances. While GAN-produced images are visually realistic, loss of fine details from inversion heavily degrades downstream performance for classification. In approaches such as SAGE, using the reconstructed image for training lowers accuracy because of the mismatch between the latent embeddings and the original information, as well as the lack of fidelity in the reconstructions. The loss of information is carried over during the editing

process, diminishing the performance of the generative augmentation. Enhanced inversion would increase both editability of the image and downstream task performance. Specifically, the study seeks to answer the following research questions:

1. Can integrating StyleFeatureEditor inverter and Feature Editor improve latent inversion fidelity and preserve image details for style-based editing within the SAGE framework?

2. Do these modifications influence the consistency, diversity, and quality of images generated in the few-shot setting, particularly for novel classes in datasets Flowers and Animal Faces?

3. How well does FeatureSAGE perform in terms of editing reliability and disentanglement in few-shot scenarios?

4. How does FeatureSAGE compare to existing editing based few shot image generation methods, AGE and SAGE, in terms of edit stability, identity preservation, and overall image quality?

1.3 Objectives of the Study

The objective of this study is to develop and evaluate FeatureSAGE, a reliable few-shot image generator framework that improves upon the SAGE framework by integrating a StyleFeatureEditor Inverter and Feature Editor

Specifically , this study's object is as follows:

a. Integrate StyleFeatureEditing to SAGE framework: integrate the inverter and feature editor pair to improve generated image quality. This aims to make a framework variant of SAGE that can be used for a more downstream (e.g. Data Augmentation) task via high reconstructability and editability without the usual tradeoff.

b. Evaluate reconstruction quality of the inverted images of Animal Faces and Flowers using FID and LPIPS.

c. Benchmark FeatureSAGE against baseline methods, specifically SAGE and AGE, to demonstrate improvements in edit stability, identity preservation, and overall image quality.

1.4 Significance of the Study

This study contributes to the few-shot image generation domain by enhancing the editability and interpretability of latent codes in the SAGE framework. By integrating the StyleFeatureEditor's encoder and feature editor, the proposed FeatureSAGE framework improves stability and consistency in attribute group manipulation under data-scarce cases which addresses the key limitation of the existing few-shot generative models.

Aside from methodological contributions, the proposed approach also has practical relevance and has several downstream applications. In fine-grained visual recognition, FeatureSAGE can generate identity-preserving variations of an animal or plant from a very few dataset, which helps train classifiers for species or breed identification in environments where collecting large dataset is impossible or difficult to acquire. In biometric work that focuses on identity, such as forensic image analysis and identity-based data augmentation, FeatureSAGE can generate multiple edited versions of an image while keeping the same person's identity. This makes the findings more realistic and more reliable. In medical and scientific imaging, researchers often have small datasets because privacy rules limit sharing, and gathering new images can be expensive. In some cases, the condition or disease is rare.FeatureSAGE generates realistic images that can serve as a synthetic dataset for training and validating machine learning models.

This study extends prior work on attribute group editing and GAN latent spaces, and it gives clear guidance for building data-efficient image generation systems that can support downstream applications that need consistent, reliable image synthesis.

1.5 Scope and Limitation

This study is limited to few-shot image generation tasks using the StyleGAN2 generator. It specifically focuses on improving the reconstruction and editability of the model by integrating the StyleFeatureEditing Encoder and utilizing the Feature Editing Module into the framework of SAGE.

The datasets used in this study are restricted to Animal Faces and Flowers which offer diverse but structured categories suitable for evaluating attribute preservation and class coherence. The study does not explore optimization-based inversion techniques or apply to GAN architectures other than StyleGAN2. Results and conclusions are therefore constrained to the SAGE pipeline with these enhancements.

CHAPTER 2 REVIEW OF RELATED LITERATURE

2.1 Generative Adevrsarial Networks

Generative Adversarial Networks (GANs) is a novel framework for training generative models via an adversarial process which was first introduced by Goodfellow et al. (2014). In the paper , "Generative Adversarial Nets" , a generator network G produces samples aimed at mimicking the data distribution , while the discriminator D estimates the probability of whether an input is real or generated (Goodfellow et al., 2014). In other words, GAN framework basically corresponds to a minimax two-player game where , a generator (G) creates samples that are intended to come from the same distribution as the training data, while the discriminator D examines the probability of the the samples provided by G being real or fake (Goodfellow, 2017).

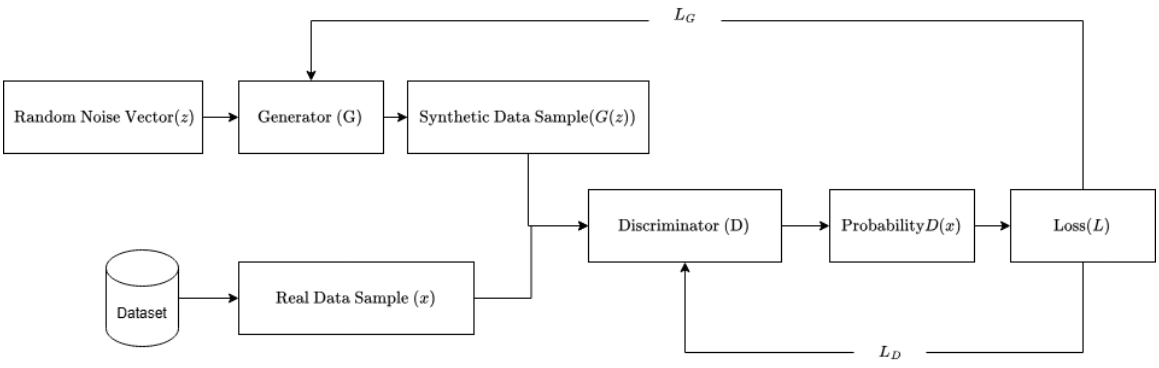

Figure 2.1 *Generative Adversarial Networks (GANs) Architecture*

As shown in Figure 2.1 the generator G takes as input a random noise vector $z \sim p_z$ (usually drawn from a simple distribution such as uniform or Gaussian) and outputs a synthetic data sample $G(z)$. The goal of G is to map the "latent" noise distribution into the data sample $G(z)$ is as realistic as possible. The discriminator D takes a data sample x as input and outputs a probability $D(x)$ estimating the likelihood that x is a

real sample rather than one produced by G (Salehi et al., 2020; Cohen and Giryes, 2022; Creswell et al., 2018).

The adversarial training process in GANs is governed by a minimax optimization. Formally, Goodfellow et al. (2014) define the value function:

$$V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}(x)}[\log D(x)] + \mathbb{E}_{z\sim p_z(z)}[\log(1 - D(G(z)))]$$

and train D and G in a two-player game:

$$\min_G \max_D V(D,G)$$

D and G are alternatively updated via minibatch stochastic gradient descent during the GAN training process. D is trained to maximize the probability of predicting whether an input from both training example and sample from G are real or fake, while G is trained simultaneously to minimize $\log(1 - D(G(z)))$ (Goodfellow et al., 2014). In reality, GAN training can be delicate: if G becomes too strong or D too weak , gradients can vanish, and if D is too strong early on , G may recieve no learning signal(saturating loss). Goodfellow et al. (2014) suggest a non-saturating heuristic for G (maximizing $\log D(G(z))$) instead of minimizing $\log(1 - D(G(z)))$ to provide stronger gradients in practice.

On its first introduction , GANs have brought about advantages compared to other generative models at the time (e.g. Variational Auto-encoders(VAE)) like sharper images, configurable sizes, richer search space, and a veratility as it can support different generator networks. However it also has its disadvatages like : mode collapse, vanishing gradients , and instability (Cohen and Giryes, 2022).

In the years following the initial discovery of Generative Adversarial Networks (GANs), there have been numerous improvements made to their architecture that have resulted in better image quality, more stable training, and expanded capabilities. Early

research highlighted the importance of convolutional structures for successful GAN performance. One influential contribution was made by Richardson et al. (2021), who introduced the Deep Convolutional GAN (DCGAN) and a set of practical design recommendations for building effective convolutional GANs. Instead of employing traditional pooling or unpooling layers, DCGAN utilized strided convolutions to decrease spatial resolution and transposed convolutions to increase it. This allowed both G and D to learn spatial features more effectively and led to outputs that were more realistic (Goodfellow, 2017). They also recommended batch normalization in almost all layers of both G and D, excluding the output and input layers respectively, which greatly stabilized training and facilitated learning proper scales.

These guidelines enabled DCGANs to synthesize high-resolution images in "one shot" and learn disentangled latent representations that enable simple vector arithmetic for changing semantic attributes of the output. DCGAN also popularized the use of Adam as an optimizer for GANs (Goodfellow, 2017). In conclusion, this research provides a comprehensive overview of how convolutional structures can improve the performance of GANs, and how practical design recommendations such as strided convolutions, transposed convolutions, batch normalization, and Adam can enable these improvements. By following these guidelines, it is possible to build high-quality image synthesis models that learn complex spatial relationships in images and produce realistic outputs. In particular, DCGANs are able to synthesize high-resolution images in "one shot" and learn disentangled latent representations that allow for simple vector arithmetic for changing semantic attributes in the output, as well as enable more flexible model configurations by allowing the use of a different optimizer, such as Adam, which has been shown to be effective in training GANs. Furthermore, DCGANs popularized the use of batch normalization across all layers in both the generator and discriminator networks, excluding the output and input layers respectively, which greatly stabilized training and facilitated learning proper scales. These findings high-

light the importance of convolutional structures in improving the performance of GANs and provide a comprehensive overview of how practical design recommendations can be used to build high-quality image synthesis models that learn complex spatial relationships in images and produce realistic outputs.

While StyleGAN and StyleGAN2 have achieved impressive results for image realism, they still face some challenges. The main limitation of StyleGAN lies in its inability to accurately imitate real-world images and in the fact that it cannot directly manipulate the latent space within the network. Additionally, even when inversion is implemented, the resulting images may not always capture identity or be structurally accurate. Despite these limitations, StyleGAN has shown promise in some unstructured scenes and faces. However, as **?** observed, StyleGAN struggles with domains that do not exhibit strong structure and may even replicate demographic biases in the training set. Finally, while StyleGAN can generate images at high resolution with ease, it requires extensive GPU resources to train on high-resolution data. Moreover, careful tuning of the architecture is also necessary to ensure fairness and accuracy. Overall, while StyleGAN has achieved significant progress for image realism, there are still challenges that need to be addressed in order to fully utilize its potential for creative and sophisticated use cases.

2.2 StyleGAN and StyleGAN2

Early GANs suffered from instability and mode collapse, which Karras et al. (2018) addressed by Progressive GANs (PGGAN): training begins at low resolution and gradually adds layers as resolution increases (Melnik et al., 2024). Building on this, StyleGAN Karras et al. (2019) proposed a novel style-based generator architecture that achieved state-of-the-art image quality and disentangled latent factors. StyleGAN is a variant of GAN that builds on its framework by radically redesigning the generator.

Instead of feeding the latent code **z** directly into the generator, StyleGAN first maps

$\mathbf{z} \sim \mathcal{N}(0, I)$ to an intermediate latent $\mathbf{w} = f(\mathbf{z})$ using an 8-layer fully-connected network (Karras et al., 2019). This **w** lives in a new "style" space $\mathcal{W}$ that is learned to be less entangled than the original latent space. Learned affine transforms then convert **w** into per-layer style parameters (y_s, y_b) that modulate each convolution via Adaptive Instance Normalization (AdaIN) (Karras et al., 2019, 2020). Concretely, for each feature map x_i in a convolutional layer, StyleGAN applies:

$$\text{AdaIN}(x_i, y) = y_{s,i} \left(\frac{x_i - \mu(x_i)}{\sigma(x_i)} \right) + y_{b,i}$$

, where y_s and y_b (scalars for channel i) are derived from **w**. In practice, StyleGAN's mapping network f (an eight-layer MLP) produces **w**, and then each convolutional layer is modulated by the corresponding (y_s, y_b) pair. This allows scale-specific control of features: coarse spatial styles (e.g. pose or overall shape) are set by early layers, while fine-grained details (e.g. color, texture) are set by later layers Melnik et al. (2024); Karras et al. (2019). Notably, the mapping network and style modulation enable style-mixing: two latent codes can be injected at different depths, blending coarse features from one code with fine details from another (Melnik et al., 2024).

Importantly, StyleGAN replaces the standard random input with a learned constant 4x4x512 tensor, and all variation comes from the style vectors and added noise. At each convolutional layer, StyleGAN also injects a random noise map (one per pixel) which is added to the features with learned per-channel weights. This stochastic noise produces high-frequency variation (e.g. hair wisps, freckles) without affecting global attributes. Together, these design choices yield unsupervised disentanglement of high-level attributes from stochastic detail: Karras et al. (2019) observe that their style-based generator "leads to an automatically learned, unsupervised separation of high-level attributes (e.g., pose and identity) and stochastic variation in the generated images". In experiments, StyleGAN achieves state-of-the-art image fidelity (low Fréchet Inception

Distance) and more linear, less entangled latent factors than conventional GANs.

The architectural differences from a standard GAN can be summarized as:

- Intermediate latent space and mapping network: StyleGAN learns a mapping f : **z** $\rightarrow$ **w** (eight-layer MLP) so that **w** in space $\mathcal{W}$ is disentangled
- Style modulation via AdaIN: Style vectors from **w** provide per-layer scale/bias that modulate each convolution by AdaIN, as in the formula above.
- Noise injection: Learned Gaussian noise is added to each layer to control stochastic details
- Learned constant input: The generator begins from a fixed learned 4x4 tensor instead of random noise
- Progressive growing: StyleGAN adopts the progressive training strategy from ProGAN [Karras et al., 2018], starting at 4x4 resolution and gradually adding layers to reach high resolution (Karras et al., 2018).

StyleGAN2 Karras et al. (2020) builds directly on StyleGAN but redesigns the generator to eliminate artifacts and boost quality. The key insight is that the original AdaIN normalization can destroy meaningful signal: normalizing each feature map breaks global correlations needed for coherent details. Therefore, StyleGAN2 "bakes" the style modulation into the convolution weights and replaces the per-feature normalization with a weight demodulation operation. In practice, this works as follows: let w_{ijk} be the convolution weight connecting input channel i to output channel j (at spatial position k). Given a style scale s_i (from **w**) for channel i, StyleGAN2 first multiplies w_{ijk} by s_i (modulation), yielding $w'_{ijk} = s_i w_{ijk}$. It then computes the output standard deviation σ_j of the convolution and demodulates by dividing by σ_j:

$$w''_{i,s,j,k} = \frac{w'_{ijk}}{\sqrt{\sum_{i'}(w'_{ijk})^2 + \epsilon}}$$

. This eliminates the need for explicit instance normalization after the convolution. The result is that each convolutional layer can apply the style s_i directly to its weights, with the demodulation automatically preserving unit variance. This modification removes the "blob" artifacts common in StyleGAN outputs and leads to crisper, more natural images.

Training in StyleGAN2 takes a different path. Instead of slowly increasing complexity, the model uses a set structure right away. Skip connections and residual blocks link features across scales, much like in MSG-GAN or U-Net designs. This setup keeps signal flow strong throughout training. Without needing to add layers over time, stability improves naturally. The approach skips the extra burden of gradual expansion. Yet, key elements remain - style mapping and noise inputs stay part of the process. These are simply embedded into the updated generator framework. Stability meets continuity here. Not every change means replacing what worked. Another finding from Karras et al. (2020) shows that applying path length regularization to the mapping network helps balance how shifts in W affect visual output. Because of this adjustment, movements through latent space translate more consistently into noticeable image differences. The result is a smoother connection between codes and images, easier reversal from image back to code, better separation of features, along with improved editing control.

The architectural improvements in StyleGAN2 can be summarized as:

- Weight demodulation instead of AdaIN: Styles modulate convolution weights and are normalized in-weight, removing all explicit feature-map normalization.

- No instance normalization layers: By design, the generator no longer uses AdaIN or other instance/batch norms; all normalization is implicit in weight demodulation

- Elimination of progressive growing: The network is trained at fixed full resolution, using skip/residual connections to preserve stability
- Path length regularization: A regularizer on the mapping network encourages a smooth, linearized latent space.

These changes yield a substantial quality gain. For example, Karras et al. (2019) report that fully updated StyleGAN2 (no progressive growing, weight demodulation, path regularization) achieves a Fréchet Inception Distance of about 3.3 on FFHQ, compared to 4.4 for the original StyleGAN baseline.

Though StyleGAN and its successor deliver remarkable visual fidelity, certain constraints remain evident. Generated outputs along with manipulation options depend strictly on what the model learned during training. Control through latent space works solely within the boundaries of synthetic imagery the system supports; actual photographs require conversion into domain via GAN inversion prior to adjustments (Bermano et al., 2022). Yet perfect preservation of subject traits isn't always achieved post-inversion. What makes face images easier for these models becomes a hurdle elsewhere - disorganized settings confuse their output. Bermano and team noted back in 2022 how StyleGAN falters when patterns lack clear repetition. Problems show up beyond performance too, since learned behaviors include social imbalances present in source data. Training runs expose another layer of difficulty: high-res workloads need powerful graphics processors, precise adjustments. Complexity piles up, even if results look effortless.

2.3 The Latent Space

Early GANs drew a latent vector z from a simple distribution and fed it through a generator, but the resulting latent space was largely entangled and hard to control. InfoGAN by Chen et al. (2016) introduced an information-maximizing objective to

encourage disentanglement, yielding interpretable latent factors. The progressive GAN (PGGAN) of Karras et al. (2018) improved image quality via multiresolution training but still used a single latent code at the input. A major shift occurred with StyleGAN Karras et al. (2019), which used a learned mapping network to transform the input z into an intermediate latent W space This mapping ,via 8 fully-connected layers, "unfolds" the simple z distribution into W, aligning it to the generator's feature distribution and enabling axis-parallel edits(Melnik et al., 2024). StyleGAN also injects this W code and a random per-layer noise at each convolutional layer, allowing layer-wise control of coarse-to-fine image attributes.

StyleGAN2 built on this by replacing Adaptive Instance Normalization (Ada in) with weight modulation and demodulation; it further regularized the latent mapping to smooth out the generator's response to latent shifts (Karras et al., 2020). This made the latent-to-image mapping more uniform and significantly easier to invert, meaning real images could be projected more reliably into W. Importantly, StyleGAN2 popularized the use of an extended latent space W^+, in which a separate 512-dim w vector is supplied to each of the 18 generator layers. In W^+, each layer can have its own style code, greatly increasing representational power: for a 1024x1024 model, W^+ is 18x512-dim. However, W^+ vectors do not come from the learned W distribution, so naive sampling can produce realistic "face-like" images but also novel patterns that are still face-like (eg. Aliens) (Melnik et al., 2024). W^+ is used mostly for reconstruction/inversion and fine edits, whereas W remains the base latent space for smooth, in-distribution sampling. Finally, StyleGAN2's style code w is further transformed, once per layer, into a channel-wise affine vector s in Style Space S, which directly scales convolutional feature maps (Melnik et al., 2024). In StyleGAN2 , each channel's style parameter in S is a single scalar, controlling only scale (no bias). Thus StyleGAN2 introduced multiple nested latent spaces: the input Z, the mapped W, the layered W^+, and the channel-wise S.

The StyleGAN architecture encourages disentanglement by design. Karras et al. (2020) observed that W becomes "much closer to the required feature distribution" than z, so that semantic edits align with simple directions. In StyleGAN the W space yields more disentangled representations than the original Z space, making long-range manipulations (e.g. aging) more robust (Shen et al., 2020). InterFaceGAN explicitly exploits this: it identifies linear hyperplanes in W corresponding to binary face attributes (smile, age, eyeglasses, etc.) and shows that moving w across these decision boundaries toggles the attribute. In StyleGAN's W space, these attributes become nearly disentangled after a linear transformation, unlike in Z.

Unsupervised methods have since discovered many such directions. GANSpace applies Principal Component Analysis (PCA) to StyleGAN2 latent codes and finds that principal components correspond to high-level edits (pose, color, environment) (Härkönen et al., 2020). emarkably, GANSpace shows that a single global PCA direction, when applied at specific layers ("layer-wise decomposition"), can control attributes from coarse (camera angle) to fine (facial features) without labels. Similarly, Voynov and Babenko (2020) developed an unsupervised optimization to extract interpretable latent directions from any GAN; they report non-trivial findings (e.g. a "background removal" direction) without any supervision. Semantic Factorization (SeFa) by Shen and Zhou (2021) provides a complementary approach by performing a closed-form eigen-decomposition of the pretrained generator's weight matrices, SeFa factors the W space into semantically meaningful directions (necklace on/off, lighting, etc.) without training or samples. Empirically, SeFa's directions often match or exceed those from supervised methods (e.g. InterFaceGAN) in manipulating specific attributes.

The disentangled latent structure of StyleGAN2 enables rich image manipulation. A common paradigm is latent traversal: given a latent code w, one perturbs it along a semantic direction and observes the image change. This can be simple as linear arithmetic like how InterfaceGAN uses $z_{edit} = z + \alpha n$ for single attribute manipulation.

More advanced controllers have been developed. StyleFlow (Abdal et al., 2021) models latent editing as a continuous conditional flow: it learns a bijective mapping in latent space conditioned on attribute values (age, pose, gender, etc.), enabling smooth attribute-conditioned sampling and editing (Abdal et al., 2021). In StyleFlow, one can fix most aspects of the face and continuously change a specified attribute like gradually make the person older with high fidelity. Similarly, StyleCLIP leverages CLIP text embeddings to find latent directions: it either optimizes a latent to match a text prompt or trains a mapper network to translate text into a direction in W or S. This yields a flexible text-based interface for latent edits ("smile", "pikachu", "gothic") without requiring explicit labeled data. (Patashnik et al., 2021).

For image editing tasks, one often needs to map a real photo into StyleGAN2's latent space. GAN inversion methods seek a latent code (in W, W^+, or even feature space F) that reproduces the input under the generator. According to Liu et al. (2023), GAN-Inversion methods suffer from fidelity-editability trade-off where optimizing in W^+ (high capacity) yields near-perfect reconstruction, but the resulting w^+ may lie outside the "safe" distribution, making edits unpredictable. Conversely, constraining to W yields more stable edits but higher reconstruction error. Modern techniques address this: encoder-based methods like pixel2Style2pixel (pSp) (Richardson et al., 2021) or encoder4Editing (e4e) (Tov et al., 2021) quickly predict an approximate W^+ code; then post-processing (latent optimization or a "pivotal tuning") refines it. For example, pSp's encoder directly outputs 18 layer-specific style vectors into W^+, inverting a real face in one forward pass. Such hybrid inversion plus fine-tuning allows one to start with an interpretable latent in W/W^+ and then apply the semantic edits learned for synthetic images.

Ultimately, StyleGAN2's latent space serves as a controllable image model. One can generate images with specified attributes by sampling z and then steering $w = M(z)$ in desired directions (Patashnik et al., 2021). For instance, adding a "smile" direction

to w yields smiling faces. More generally, the latent allows fine-grained tuning: by selecting w^+ values per layer, one can combine styles from different images or enforce multi-attribute conditions. StyleGAN2's latent space, especially W and S , provides a semantically meaningful coordinate system. Its evolution from earlier GAN latents (simple Z) to this multi-space structure underpins the state-of-the-art in GAN-based image editing and disentangled generation.

2.4 Attribute Editing in GAN Latent Spaces

Modern Generative Adversarial Networks (GANs) have shown that the latent codes controlling image generation often embed rich semantic structure. In particular, many high-level attributes (e.g. age, smile, pose) correspond to almost-linear directions in the latent space (Shen et al., 2020; Härkönen et al., 2020). Once a direction vector d for an attribute is found , editing is done by simple latent arithmetic $w_{edit} = w+\alpha d$ where w is the original latent code α is a scalar step size, and d is the learned attribute direction. For example, InterFaceGAN treats attributes as binary labels and trains a linear SVM in the latent space; the normal vector of the separating hyperplane becomes d (Shen et al., 2020). Style-based GANs (StyleGAN1/2) enhance this by using a mapping network to produce an intermediate latent W space that is empirically more disentangled than the input Z space (Karras et al., 2020; Wu et al., 2020).

There are two main approaches to finding attribute directions. Supervised methods use labeled data: for a given attribute one can train a linear classifier (e.g. on whether images have glasses) on a set of latent vectors. The classifier's weight vector (normal to the decision boundary) becomes the edit direction. InterFaceGAN demonstrated this works for many facial attributes (Shen et al., 2020). There are two main approaches to finding attribute directions. Supervised methods use labeled data: for a given attribute one can train a linear classifier (e.g. on whether images have glasses) on a set of latent vectors. The classifier's weight vector (normal to the decision boundary) becomes the

edit direction. InterFaceGAN demonstrated this works for many facial attributes. Unsupervised methods do not require labels: GANSpace, for instance, collects many random latents and applies Principal Component Analysis to find dominant directions. These principal axes often correspond to pose, lighting, or object type. Other factorization techniques, like SeFa, and style-space channel-searchers similarly find promising axes with little or no supervision (Shen et al., 2020; Härkönen et al., 2020; Shen and Zhou, 2021).

Once a direction d is chosen, editing is simply walking along that direction in latent space. Concretely, after inverting or sampling a code w, one produces an edited image by generating from $w + \alpha d$ for various α. InterFaceGAN likewise shows that adding the gender vector shifts a face's apparent gender. In general , Attribute Editing Models use the same principle of finding d or Δw and setting $w_{edit} = w + \alpha d$ to change attributes in an image (Härkönen et al., 2020; Shen et al., 2020; Li et al., 2023; Wu et al., 2020)

Importantly, these methods rely on latent disentanglement: the idea that different factors are (at least partly) separable. StyleGAN's design helps here: its intermediate latent W is empirically more disentangled than the raw Z space (Wu et al., 2020). StyleGAN2's per-channel style parameters ("StyleSpace") have been shown to control very localized attributes (e.g. hair color, background elements) with minimal interference. In contrast, earlier GANs had more entangled latents, making clean edits harder (Salehi et al., 2020; Creswell et al., 2018).

However, disentanglement is not perfect. Most discovered directions still have residual cross-attribute effects. For example, one edit vector might change both "smile" and "age" together. This entanglement is noted in the literature: many control directions affect multiple attributes and can be "non-local" (Wu et al., 2020). To mitigate this, some methods perform conditional manipulation: e.g. subtracting out the projection of one direction on another to isolate each effect. But instability remains when editing multiple attributes at once, often producing unrealistic images if

not handled carefully.

Attribute editing assumes that one has latent code w to edit . To apply editing to real photos, a critical step is GAN inversions. By finding a latent code that faithfully reproduces a given real image, one can apply learned semantic manipulations ,adjusting age, hairstyle, or pose, to that image (Bobkov et al., 2024; Alaluf et al., 2021). We find a latent code z^* whose generator output matches the target image. Once z^* is found , one can apply the w_{edit} formula to manipulate the image attributes. According to Xia et al. (2022), there 2 gan inversion methods. Namely, the learning-based(encoder-based) and optimization based Gan inversions.

- Optimization-based inversion solves for z^* by backpropagation: it iteratively adjusts z to minimize reconstruction loss $||G(z)-x||$ This often yields high-fidelity reconstructions but can be slow and may get stuck in local minima.

- Learning(Encoder)-based trains a neural network $E(\cdot)$ to map images to latent codes. A trained encoder can invert images quickly, though it must be carefully regularized to align with the pretrained generator. Hybrid approaches use an encoder for an initial guess followed by optimization refinement (Xia et al., 2022).

GAN inversion is what allows editing of real images. For instance, the survey by Xia et al. (2022) illustrates how an inverted code z^* can be altered by adding attribute vectors n_1, n_2 to change the age or smile of a real face. They note that after inversion, the same latent-space editing techniques apply "without requiring any ad-hoc supervision" However , inversion quality has limits: early models like Progressively Growing Gan (PGGAN)(Karras et al., 2018) were hard to invert faithfully, whereas StyleGAN's architecture yields much better inversions (Karras et al., 2020). Imperfect inversion can blur details or introduce artifacts, which in turn limits how well edits carry over to the final image.

2.4.1 Attribute Group Editing

Beyond single images, recent work uses latent editing for few-shot image generation: synthesizing new images of a novel category given only a few examples. The idea is to treat each new class as a 'set of attributes' and apply the learned edit directions to produce more samples. A pioneering approach is Attribute Group Editing (AGE) (Ding et al., 2022). AGE assumes that images are combinations of category-relevant and category-irrelevant attributes. It represents a category by its class embedding w_c: the mean latent of the few examples. The attributes that define the category (e.g., animal shape) are captured in w_c. AGE then learns a sparse dictionary of category-irrelevant attribute directions (e.g. fur color, pose) by doing sparse coding on the differences $w_{sample} - w_c$ for many samples. The key assumption is that an attribute direction (such as 'smiling' or 'red flower') is shared between categories. To generate new images of an unseen category, AGE starts with the class embedding w_c and adds combinations of these dictionary directions, creating diverse variations while keeping the features of the core category fixed. This editing-based generation requires no GAN retraining and can produce high-quality, diverse images even from one or few inputs.

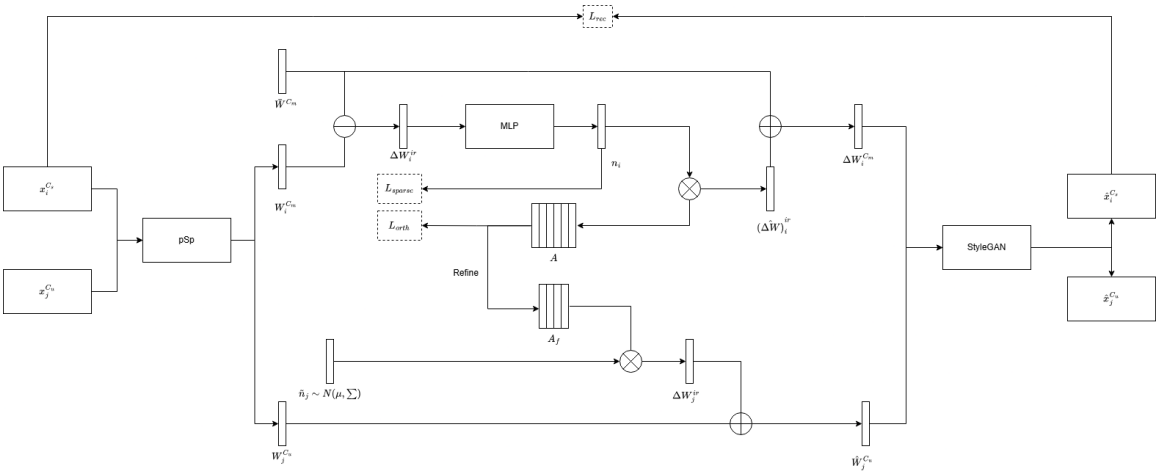

Figure 2.2 *Attribute Group Editing Framework*

AGE provides a powerful framework, but has some limitations. The SAGE (Stable

AGE) follow-up aims to improve class consistency. Instead of editing from just one example, SAGE uses all a few-shot images to robustly estimate the class embedding. It leverages the same attribute dictionaries but selects appropriate directions based on the new class's similarity to the seen classes. In effect, SAGE 'injects the whole distribution of the novel class into StyleGAN's latent space', preserving the identity of the category. This leads to more stable outputs that help downstream tasks (e.g., classification) perform better on the generated images (Ding et al., 2023).

Most recently, TAGE (Trustworthy AGE) has extended this line further. TAGE also uses a codebook (dictionary) of attributes, but introduces transformer-style modules. Its Codebook Learning Module (CLM) finds semantic directions for category-related and unrelated attributes and builds a sparse dictionary. A Code Prediction Module (CPM) then uses global combinatorial reasoning (long-range attention) to predict the best edit codes for a given few-shot class, improving diversity and accuracy. Crucially, TAGE adds a Prompt-Driven Semantic Module (PSM): it generates textual or semantic prompts that are injected into transformer layers, guiding fine-grained editing (Zhang et al., 2024).

2.4.2 Stable Attribute Group Editing

Stable Attribute Group Editing (SAGE) was proposed as a method to overcome such limitations of AGE. Instead of editing from a single exemplar, SAGE initially employs all the given few-shot images to estimate a stable class-center embedding. It does so by capitalizing on the pretrained category-relevant attribute dictionary (learned from visible classes) to linearly represent the novel class centroid in the latent space. As described by Ding et al., since there are not a sufficient number of examples to calculate a credible mean directly, "the category-relevant dictionary… is complete enough to cover the unique attributes of any related categories," a linear combination of the dictionary atoms can "recover the class-specific features of the unseen classes" (Ding

et al., 2023).

In practice, SAGE projects the few-shot samples onto such an attribute dictionary to separate their shared class features from each other, essentially removing the attributes not related to categories and retaining the attributes that are related. According to the authors, such a disentanglement also helps the model to estimate a more accurate class embedding even for a very small number of samples. The experiments also indicate that the estimated embedding from only three face images well approximates the ground truth centroid over hundreds of actual instances.

In the same paper, the weights from the projection are utilized for adaptively choosing which category-irrelevant editing directions to use. Taking cues from biological taxonomy, the authors make the assumption that categories sharing the same relevant characteristics which is being members of the same genus, for example they also share compatible and safe irrelevant characteristics. Consequently, SAGE picks the category-irrelevant directions from the highly similar seen classes depending on the projection coefficients. The approach enables the method to introduce the distribution of the unseen class to StyleGAN's latent space by essentially repositioning the generated sample cloud towards the actual class manifold. Anchoring the synthesis about this reconstructed embedding and then carrying out filtered edits, SAGE manages to maintain class identity despite generating diverse and realistic variations.

This refinement of AGE yields clearer gains in category consistency and diversity. Quantitative results show that SAGE produces samples with significantly better quality (lower FID) and greater diversity (higher LPIPS) than AGE and competing few-shot methods. For example, without retraining the GAN, "SAGE generates images with higher quality and diversity" than prior fusion- or transformation-based approaches, and its class-embedding mapping makes it "much more stable than AGE". Qualitatively, Ding et al. illustrate that AGE could produce bizarre artifacts (a "cat with a dog nose" or a "bird with two heads") when editing from extreme inputs, whereas SAGE's tailored

embedding and adaptive edits yield realistic variants without such errors. In the latent space of an unseen class, AGE's few-shot samples often lie at extreme positions and its generated points cluster narrowly around them. By contrast, SAGE's injections form a much wider latent cloud that more closely matches the true distribution. The authors summarize that SAGE "relocates the whole distribution of the novel class in the latent space, thus effectively avoiding category-specific editing and enhancing the stability". Overall, Ding et al. report that SAGE achieves "more stable and reliable few-shot image generation" with significantly better category retention than previous methods.

Nonetheless, SAGE does not solve all challenges. The authors note that its generative range is still more restricted than that of real data: the "dynamic range" of editable variations in SAGE is smaller than in the true class. In practice, SAGE currently uses a uniform editing intensity for all directions (since estimating per-attribute scales from few samples is infeasible) and its dictionary contains only attributes that recur across the seen classes. Consequently, some subtle class-specific variations may still be missing. As the paper observes, "SAGE simply assumes a unified intensity for editing direction sampling," and capturing attribute-dependent editing amplitudes or class-specific edits remains an open issue. Moreover, while classification performance is improved over AGE, it still generally lags behind a model trained on real examples. This gap – and the reliance on a pretrained StyleGAN inversion as the generator – highlight future work: extending SAGE's ideas to handle multi-scale attributes and other generative models may be needed to fully close the gap for few-shot tasks.

2.4.3 Datasets and Applications

Attribute editing techniques are typically developed and evaluated on diverse image domains. The datasets Animal Faces HQ and Flowers are common: they have many labeled attributes and StyleGAN models are readily available for them. Few-shot generation research has also expanded to other domains. For instance, the Flowers dataset

has 102 flower species (Gupta et al., 2024), and Animal Faces (or AnimalFacesHQ) has 149 animal species.

Latent editing has practical use in data augmentation (few-shot generation) and interactive image manipulation. For example, one can start with a single animal or flower image and synthesize variants by editing color, pose, or environment using learned directions. However, note that many attributes do not directly map across domains; cross-domain transfer of semantic concepts remains challenging.

2.4.4 Attribute Editing limitations and challeges

Despite their promise, latent-space attribute editing methods face several challenges (Wu et al., 2020; Ding et al., 2022, 2023; Shen et al., 2020; Xia et al., 2022; Liu et al., 2023):

- Attribute entanglement. Discovered directions often change multiple features at once. For instance, moving along an "age" vector might also alter "expression". Entangled edits require careful design (e.g. subtracting projections) or more sophisticated disentanglement techniques. Fully disentangling a GAN's latent space is still an open problem.

- Multi-attribute interactions. Applying more than one edit vector can be unstable. Sequential edits may accumulate artifacts, and combining directions (e.g. "add glasses" and "increase smile") can produce unrealistic results if the attributes interact in complex ways. Ensuring consistent multi-attribute editing (for example, editing several facial features simultaneously without distortion) remains tricky.

- Inversion quality. Editing real images relies on GAN inversion. If the inversion does not perfectly capture the original, edits may fail or degrade fidelity. Early GANs (like PGGAN) struggled with inversion.While StyleGAN produces better inverted results, truly lossless inversion is hard. Any residual reconstruction error limits how cleanly attributes can be edited.

- Generalization to diverse datasets. Most latent-editing research has centered on well-studied domains (faces, common objects). Extending these methods to fine-grained categories like birds or flowers raises issues. The learned GAN (or its latent semantics) may not capture useful directions in these domains, especially if attribute labels are scarce. Additionally, attributes meaningful in one domain (e.g. "nose shape" in faces) do not translate to others. New methods must learn or adapt directions that make sense for non-face classes.

2.5 GAN Inversion

GAN inversion refers to finding a latent code in a pretrained GAN (here StyleGAN2) that reproduces a given real image when passed through the generator. In practice, one inverts an image x by solving for a style code w such that the StyleGAN2 generator G outputs an image $G(w) \approx x$ (Tov et al., 2021). This can be done via optimization (iteratively adjusting w for each image, which is slow) or via a learned encoder (a neural network that predicts w in one forward pass) (Alaluf et al., 2021; Tov et al., 2021). StyleGAN2 uses a mapping network from a base Gaussian z into an intermediate latent space W, which is more disentangled. Many inversion methods actually use an extended latent space W^+ (one 512-D style vector per layer) for greater expressiveness (Richardson et al., 2021; Tov et al., 2021; Bobkov et al., 2024). Once an image is inverted to a latent code, one can edit the image by manipulating that code (e.g. adding learned attribute directions or swapping style layers) and generating the edited image via G. High-quality inversion thus requires balancing reconstruction fidelity (low distortion between x and $G(w)$) with with editability (the code lies in a region where semantic edits are meaningful).

How GAN inversion works (Richardson et al., 2021; Alaluf et al., 2021; Tov et al., 2021):

- Pretrained StyleGAN2: Use a generator G trained on a target domain. Fix its

weights.

- Latent spaces: Identify the latent space to invert into (typically W or extended W^+ sometimes an additional feature space F).

- Encoder or optimization: For encoder-based inversion, train a feed-forward network E to map images to latents. For optimization-based inversion, for each image x optimize W by minimizing a loss $\|x - G(w)\|$. Encoder methods are much faster (one pass) but historically had worse fidelity than optimization.

- Architecture of encoders: Typically, an encoder is a convolutional backbone (often ResNet) that produces latent parameters. For example, pixel2style2pixel (pSp) uses a ResNet + feature pyramid, producing style vectors via map2style blocks for each scale. Encoder4Editing (e4e) is similar but outputs one base style plus per-layer offsets. ReStyle applies such encoders iteratively, and StyleFeatureEditor uses a two-branch encoder for both W^+ and feature-latents.

- Loss functions: Encoders are trained to minimize a combination of losses on reconstruction. Common losses are pixel-wise L2, perceptual (LPIPS) loss, and often an identity or feature loss to preserve semantics (e.g. using ArcFace or self-supervised features).Many methods also add regularization: for example, e4e adds L2 penalties on offsets to keep the final code near W, and some use adversarial (latent discriminator) loss to keep codes within the true latent distribution.

- Inference and editing: At test time, given image x, the encoder predicts a latent code (or code+offsets or code+features). This code is fed into the fixed generator G to reconstruct x. For editing, one modifies the latent (e.g. by style mixing or adding semantic direction vectors) and uses G to synthesize the edited image.

2.5.1 pixel2style2pixel (pSp)

pSp is an early StyleGAN2 encoder (CVPR 2021) that maps an image to the extended latent space W^+. It uses a ResNet-50 backbone with a feature pyramid producing three scales of feature maps. Each scale is fed through small map2style networks to generate 512-D style vectors corresponding to StyleGAN's layers. The output is a full W^+ The encoder is trained with a weighted sum of losses: pixel L2, LPIPS perceptual loss, and an identity loss (ArcFace for faces, or a self-supervised ResNet for other domains). A small regularization term on the styles can also be used. By design, pSp prioritizes reconstruction fidelity. It "directly maps a real image into the W^+ latent space with no optimization required". In experiments it achieves low LPIPS and MSE reconstruction error. Because it predicts full W^+, pSp can accurately capture fine details of the image. After encoding, image editing is done by standard StyleGAN manipulations (e.g. adding latent offsets or style-mixing). For example, pSp naturally supports style-mixing for multi-modal synthesis: one can replace selected style vectors with those from a random code to generate variations.

pSp is fast (single forward pass) and yields high-quality reconstructions. It has been shown to preserve identity and details (e.g. hairstyle, lighting) better than many earlier methods. It also generalizes to various image-to-image tasks without retraining (e.g. frontalization, super-resolution) because of the rich StyleGAN latent space.

However , since pSp uses full W^+, the encoded latents can lie far from the original distribution learned in W. This can degrade "editability": edits in regions of W^+ far from the data manifold may produce unrealistic results. pSp's very high reconstruction fidelity sometimes comes at the cost of less coherent edits, compared to encoding into W or constrained spaces. (Richardson et al., 2021)

2.5.2 Encoder4Editing (e4e)

Encoder4Editing (e4e, ICCV 2021) extends pSp by explicitly addressing the reconstruction editability trade-off. The key idea is to keep the inverted latent closer to the original W distribution even while allowing high-fidelity reconstruction. Architecturally, e4e still uses a ResNet+feature pyramid encoder, but instead of predicting all style vectors independently, it predicts one base style code w_0 and a small offset vector o_i or each of StyleGAN's 18 layers. The final style for layer i is $w_0 + o_i$.During training, two principles are enforced:

1. Progressive training: initially predict only w_0 then gradually allow one layer's offset at a time, progressing from coarse to fine layers.

2. Offset regularization: penalize the magnitude of the offsets (via L2) to keep $w_0 + o_i$ close to w_0 and thus close to the learned W manifold. A small latent discriminator (GAN loss in latent space) is also trained to distinguish real W samples from the encoder's outputs.

During training e4e uses the same image-space losses as pSp (L2, LPIPS, identity) and adds the above editability losses. The result is an encoder that strikes a balance: it produces reconstructions nearly as good as pSp, but the codes are "closer to W" so that semantic edits transfer more reliably. e4e still outputs an extended style code, but with built-in bias towards W. In practice one can edit e4e codes using the same methods (adding latent directions, style mixing), and edits tend to be more robust. For faces, e4e also incorporates a cosine identity loss (ArcFace or general ResNet) to preserve identity under reconstruction.

e4e greatly improves editability. It was shown to maintain coherent attribute manipulations (age, expression, pose) on real images better than pSp, with only a minor increase in reconstruction error. ts inference speed is still one forward pass (plus negligible overhead from the latent discriminator at training time). It generalizes to

multiple domains (faces, cars, etc.) by using a domain-agnostic feature extractor for identity.

To achieve editability, e4e accepts a small drop in reconstruction fidelity. Very fine details may be slightly lost compared to pSp. Also, the progressive training and latent GAN add complexity, and tuning the trade-off weights requires care. The notion of "proximity to W " is somewhat loosely defined , so extremely out-of-distribution images (e.g. non-human faces) may still not be perfectly encoded. Tov et al. (2021)

2.5.3 ReStyle

ReStyle (ICCV 2021) focuses on improving inversion accuracy by iterative refinement. Instead of a single encoder pass, ReStyle's encoder is applied multiple times in a feedback loop. Starting from an initial latent (the average w vector), ReStyle repeatedly concatenates the original image with the generator's current reconstruction and feeds them into the encoder. At each iteration t, the encoder predicts a residual $\Delta w^{(t)}$ to add to the current latent.

$$\text{At step t,} w^{(t)} = w^{(t-1)} + E(x, G(w^{(t-1)}))$$

,

with $w^{(0)}$, set to the average latent, and $G(w)$, the current output image. The encoder is trained to minimize reconstruction loss at each step (summing L2+LPIPS over all iterations). ReStyle can be applied on top of any base encoder (e.g. pSp or e4e). In implementation, the authors actually simplify those encoders by using only the final feature map (no pyramid). Each iteration uses a fresh forward pass through the same network (weights are shared across steps). The training, therefore, unrolls a small multi-step network with losses on each reconstructed image.

ReStyle yields a more accurate latent code after its iterations, meaning $G(w)$ more closely matches the input. This generally improves any downstream editing (since the

base reconstruction is better). In other words, it aims to match the quality of per-image optimization (but much faster). The final code is in W^+ and can be edited normally.

ReStyle significantly improves reconstruction quality compared to standard feed-forward encoders. It achieves near-optimization accuracy with only a small inference-time cost (typically 3-5x forward passes). Because it uses the same encoder architecture at each step, it avoids complex modifications. It can even be viewed as a learned optimization in latent space.

The iterative loop means inference is slower (several passes). Although small (e.g. 5 steps), it is no longer a single-shot encoder. Also, ReStyle does not explicitly enforce editability constraints - it focuses on fidelity. It inherits the latent space choice of the base encoder: if initialized in W^+ the result can still lie outside the original W distribution. Finally, training is more complex (unrolling multiple steps with deep supervision) and may be sensitive to number of iterations. (Alaluf et al., 2021)

2.5.4 StyleFeatureEditor

StyleFeatureEditor (SFE) introduces a new latent representation combining StyleGAN's usual style codes, $worw_i$, with a feature tensor F_k from an intermediate generator layer. Intuitively, the feature tensor F_k , high-dimensional spatial features , captures fine detail that W^+ alone may miss. The method has two networks: an inverter I and a feature editor H. The inverter I takes an image X and predicts both a style code $w \in W^+$ and a feature map F_k. Specifically , I first maps X to w via linear layers, and also to an intermediate feature F_{pred} . Then it fuses F_{pred} with the actual generator's feature. to produce the final inversion feature F_k.

Crucially, StyleFeatureEditor trains a separate editor network $H(F, \Delta)$ that adjusts the feature tensor for editing. Given an editing direction d in n latent space, one computes the change $\Delta = F_k(w + d) - F_k(w)$ by sampling from a pretrained encoder. Then H is trained to take the inverted feature F_k and Δ and output the edited feature

$F'_k = H(F_k, \Delta)$. In inference, for a desired edit d, one computes $w' = w + d$, uses H to updated $F \Rightarrow F'_k$ and generates the edited image from w', F'_k.

StyleFeatureEditor undergoes 2 phase training:

1. Inverter I is trained on real images with standard losses (L2+LPIPS+identity, plus latent regularization and adversarial loss).

2. Editor H is trained on synthetic pairs: for each real X, a pretrained W^+ encoder produces w, an edit d is sampled, yielding, X'_E by adjusting H(inverter frozen).

StyleFeatureEditor achieves extremely high reconstruction quality and preserves detail even under editing. In experiments it produces nearly indistinguishable inverted images, and crucially it "preserves most" of those fine details when editing. By operating directly on feature maps, it can maintain textures and fine structure that pure latent edits often lose. It is capable of editing challenging out-of-domain examples and does not rely solely on W or W^+ for content. (Bobkov et al., 2024)

2.6 Few-Shot Image Generation in GANs

Few-shot image generation (FSIG) extends few-shot learning to generative modeling: the goal is to synthesize novel images for a new category given only a handful of training examples. This contrasts with few-shot classification, where techniques like transfer learning have proven effective (Chowdhury et al., 2021). In FSIG, however, naively fine-tuning a powerful generative model on few examples leads to overfitting and mode collapse. According to Zhao et al. (2023), modern GANs "tend to replicate the training samples" when adapted to only 10 target images. As a result, FSIG methods typically transfer rich prior knowledge from a large source domain and then adapt carefully to the new domain. This is often framed as a style-transfer or domain-adaptation problem: a generator pretrained on a large dataset ,like FFHQ faces, is fine-tuned on a small target set without access to the source data (Cao and Gong, 2024; Zhao et al., 2023). The

key challenge is preserving the source model's diversity and high-fidelity output while learning the new style from very limited data.

Generative Adversarial Networks (GANs) have been the dominant framework for FSIG. The standard paradigm is to take a GAN (often StyleGAN or BigGAN) pretrained on a large source dataset and adapt it to the new class. Early approaches simply fine-tuned all weights on the few-shot target (TGAN) (Zhao et al., 2023), but this quickly overfits (the generator memorizes the few examples). Numerous strategies have been proposed to prevent collapse and preserve diversity during adaptation. or instance, FreezeD freezes some high-resolution layers of the discriminator, ADA (adaptive data augmentation) introduces random augmentations during training, and EWC applies an Elastic Weight Consolidation penalty on important weights (Mo et al., 2020). Another approach Cross-domain Correspondence (CDC) preserves the relative distances between different source-domain samples in the target embedding space (Ojha et al., 2021). In practice, Zhao et al. (2022) found that most of these GAN-adaptation methods converge to similar image quality after many iterations – the real difference is how slowly they lose diversity during training (Zhao et al., 2023). In other words, diversity degradation (mode collapse) is the main bottleneck: methods that slow down this collapse (e.g. by freezing parameters or using strong regularization) tend to perform better overall.

Building on this insight, Zhao et al. propose a contrastive learning scheme to retain diversity. Their Dual Contrastive Learning (DCL) maximizes mutual information between the source and target generators: the idea is that the same latent input z should map to semantically corresponding images before and after adaptation. Concretely, they use a contrastive loss on multi-level features of the generator and discriminator to enforce that source and target outputs remain similar in high-level content. This preserves the source generator's rich variation in the adapted model, yielding state-of-the-art results on several datasets (Zhao et al., 2023). Similarly, Hong et al. (2020)

introduce DeltaGAN, which decomposes generation into a reconstruction sub-network (that computes an intra-class "delta" transformation between two samples of the same category) and a generation sub-network (that applies a sample-specific delta to a base image) (Hong et al., 2022). DeltaGAN's adversarial "delta matching" encourages high diversity by learning many distinct transformations from the limited examples.

GAN-based FSIG methods share a transfer-learning theme: leverage a pretrained model's knowledge and carefully regularize the adaptation. Typical training strategies include fine-tuning, parameter freezing, data augmentation, and contrastive or distance-preserving losses. Their success is usually measured by metrics like Fréchet Inception Distance (FID) or LPIPS (perceptual diversity).

Transfer learning is a commonly used training strategy in generative models that starts from a pre-trained model (GAN, VAE, or diffusion) and updates it using a few target images. Pure fine-tuning can often overfit the model, making it difficult to generalize well (Zhao et al., 2023). Meta-learning involves training on simulated few-shot tasks so that it can adapt quickly. Contrastive/Distance Losses are used to enforce that the adapted generator preserves relationships between different domains. Data augmentation and synthesis create pseudo-examples by replacing source images with those in the target fine-tuning process. While some GANs/VAEs may use MAML or Reptile, most often they still need per-task fine-tuning and struggle to generate sharp images (Hong et al., 2022). In contrast, meta-learned GANs/VAEs have been attempted but are usually limited by their overfitting problem (Hong et al., 2022). Meta-Learning involves training on many simulated few-shot tasks so that it can adapt quickly. Data augmentation and synthesis create pseudo-examples by replacing source images with those in the target fine-tuning process. In this way, humans can better understand how to generate realistic images based on the data they have available.

Evaluation strategies are also noteworthy. FSIG is evaluated on cross-domain tasks such as generating artistic or cartoon versions of FFHQ faces, or adapting LSUN Church

to "Haunted House" style, or similar sketch-to-photo tasks. Metrics include FID for quality (when enough data exists) and LPIPS or recall for diversity. Zhao et al. (2023) emphasize that with only 10 real images, FID is unstable, so they instead use intra-LPIPS (average pairwise LPIPS of generated set) to quantify diversity .

Despite these advances, few-shot image generation remains challenging. Surveys note that learning from very few or zero examples is still "a serious challenge" (Song et al., 2022). Key open issues include: catastrophic forgetting (the model loses the source distribution entirely), mode collapse (outputs lack diversity), and evaluation (how to measure quality/diversity with tiny real sample sets). Overfitting to the few examples is ubiquitous: even with strong regularization, generators tend to memorize salient features. Furthermore, different generative families have trade-offs – GANs are fast and high-quality but brittle, VAEs are stable but blurrier, and diffusion models are powerful but computationally heavy. A brief comparison: GANs excel at sharp images but require careful balancing of losses; VAEs naturally encourage covering the whole distribution (which aids diversity) at the cost of some sharpness; diffusion models sample iteratively and often capture fine detail, but naively adapting them can be even more parameter-sensitive than GANs (Cao and Gong, 2024).

CHAPTER 3 METHODOLOGY

3.1 Research Design

This paper, "FeatureSAGE: Stable Attribute Group Editing With Integrated StyleFeatureEditing Encoder for Enhanced Few-Shot Image Generation," adopts a quantitative approach with an experimental research design to evaluate improvements to the existing SAGE framework. In particular, we integrate the GAN inversion module of SAGE with the StyleFeatureEditor (SFE), incorporating its Feature Editor module to enhance latent inversion and attribute editing functionality under a one-shot setting, where attribute manipulation is conditioned on a single reference image per test instance. The research employs a quantitative methodology using established automated metrics for objective evaluation. Reconstruction quality is assessed using Fréchet Inception Distance (FID) and Learned Perceptual Image Patch Similarity (LPIPS).

3.1.1 Experimental Setup

The experimental setup is designed to evaluate the effectiveness of the proposed FeatureSAGE framework in comparison to the existing SAGE method. The evaluation follows a quantitative, experimental research design that involves controlled changes to a single independent variable and the measurement of multiple dependent variables using standardized metrics.

- **Independent Variable**
  - GAN Inversion Method: The primary manipulation involves substituting the original Pixel2Style2Pixel (pSp) encoder used in SAGE with the proposed StyleFeatureEditor (SFE) encoder, which includes an integrated feature editing module for enhanced semantic attribute control.

- **Dependent Variable** To objectively evaluate the effectiveness of the GAN inversion and editing pipeline, the following metrics are employed:
  - Reconstruction Quality
    1. Fréchet Inception Distance (FID): Measures distribution-level similarity between generated and real images.
    2. Learned Perceptual Image Patch Similarity (LPIPS): Measures perceptual similarity between original and reconstructed images based on deep features.
- **Controlled Variables**
  1. Generator Architecture : A fixed, pretrained StyleGAN2 generator is used in both configurations for consistent image synthesis capabilities.
  2. Training Dataset : Both configurations use the same datasets (FUNIT Animal Faces and 102 Flower) with the same preprocessing and data-splitting.
  3. Evaluation Protocol : Use the sample unseen categories from the train-test spliting data for few-shot testing in both configurations. Evaluation is conducted using identical random seeds, few-shot sample counts (e.g., 1-shot, 5-shot), and query sets.
  4. Loss Functions : Reconstruction and editing losses are applied consistently across both systems.
  5. Optimization Settings : Both models utilize the same Optimizer type, Learning rate , Batch size and Training epochs
  6. Evaluation Metrics : All quantitative metrics (FID and LPIPS) are computed using the same implementation and parameters for consistency.
  7. Hardware & Software Environment : All experiments are conducted using

the same hardware and PyTorch environment to avoid performance inconsistencies.

- **Experimental Conditions**

  Two experimental configurations are used:

  - SAGE (Baseline): Employs the original pSp encoder with StyleGAN2.
  - FeatureSAGE (Proposed): Integrates StyleFeatureEditor (SFE) encoder and a Feature Editor.

- **Hardware specifications**: All experiments were conducted on a virtual machine instance with GPU pass-through, hosted on institutional cloud infrastructure:

  Listed below are the hardware specifications of the device that will be used for training:

  - GPU : NVIDIA GRID V100D-32A (virtualized Tesla V100) 16 GB HBM2
  - CPU : QEMU Virtual CPU version 2.5+ 2 processors @ 2.19 GHz
  - RAM : 32.0 GB

3.1.2 Few-Shot Setting Experimental Setting

In this study, the term *few-shot* refers to the number of reference images available per target class (or identity) used to construct class-level latent representations for attribute group editing and downstream evaluation. A **1-shot** setting utilizes a single reference image per target class, representing an extreme data-scarce scenario in which attribute editing and inversion must be performed with minimal visual information. This one-shot configuration is applied consistently across all datasets and experimental conditions for both the baseline SAGE and the proposed FeatureSAGE framework. By explicitly controlling the evaluation to a single reference image per class, the experimental protocol enables a systematic assessment of FeatureSAGE's

robustness, stability, and generalization capability under severe data limitations. The few-shot constraint is applied during test-time adaptation and evaluation on unseen classes, while the StyleGAN2 generator and attribute directions are learned from large-scale seen-category data.

3.2 Data Preprocessing

3.2.1 Data Sources

This research draws from a range of publicly available datasets with a varied scope of visuals such as animal faces and flowers. The datasets are popular for generating and attribute-editing applications because of their quality and well-labeled categories. For the purpose of this research's emphasis on few-shot image synthesis, only a minority of the images from each dataset are employed. Specifically, a minimal set of representative samples per class are sampled to replicate the few-shot learning situation. The datasets used are:

1. **FUNIT Animal Faces Dataset**

Figure 3.1 *Sample images from the FUNIT Animal Faces dataset. Source: (Liu et al., 2019a)*

Figure 3.1 shows representative samples from the FUNIT Animal Faces dataset. The dataset is comprised of 117,484 512 × 512 resolution high-quality animal face images and is organized in 149 classes (each class is one animal species). Each domain includes a diverse range of species and breeds, ensuring variation in appearance while maintaining consistent facial alignment to support generative model training. The images were curated from openly available sources and

standardized in resolution and alignment, with the eyes centrally positioned to provide consistency for learning. A subset of the images is reserved as a test set, with the remainder used for training. The diversity and quality of the FUNIT Animal Faces dataset make it a strong benchmark for generative and attribute-editing models in few-shot settings (Liu et al., 2019a).

2. **102 Flower (Flowers)**

Figure 3.2 *Sample images from 102 Flower Dataset. Source (Nilsback and Zisserman, 2008)*

Figure 3.2 shows sample images from the 102 Flower dataset, which comprises 102 flower classes present in the United Kingdom. There are between 40 and 258 images in each class. The flowers covered by the dataset are regularly occurring species in the UK. The dataset shows a great deal of variation in the sizes, pose, and lighting conditions of the flowers, which is an added complexity for the task of classification. Moreover, there are large intra-class differences for some categories, and there are many categories that are visually extremely similar to each other. All such attributes make the dataset extremely suitable for fine-grained visual categorization. The dataset was also used for analysis and visualization using isomap dimensionality reduction based on shape and color descriptors, which revealed the structure of the different flower categories. As for the sizes of the images, the raw images in the dataset are of varying sizes. For instance, one raw image from the dataset measures 500×667 pixels (Nilsback and Zisserman, 2008).

3.2.2 Data Preprocessing Steps

For all datasets, we will apply the following preprocessing steps:

- Resizing and Normalization: Images will be resized to 256×256 for efficiency (and 512×512 for high-res experiments) and pixel values will be normalized to the range [-1,1] for input to StyleGAN2.
- Filtering: We will remove any extremely low-quality or mislabeled images.
  - For FUNIT Animal Faces, use a face detector, MTCNN (Multi-Task Cascaded Convolutional Networks), to crop and horizontally align faces so the eyes are leveled.
- Train-Test Splitting : We assess our approach on four standard few-shot image domains, partitioning each into "seen" classes for model fitting and "unseen" classes for testing:
  - **Animal Faces.** Out of the total categories,119 are used as seen classes for training, while the remaining 30 serve as unseen classes for evaluation.
  - **Flowers.** We assign 85 categories to the training (seen) split and reserve 17 categories for the test (unseen) split.
- Augmentation: No data augmentation will be used beyond centering and flipping, since we focus on the evaluation of given domains. All datasets and preprocessing pipelines are documented for reproducibility.

3.3 Model Architecture

This study proposes a modified SAGE framework named **FeatureSAGE**, which integrates several modules to enhance few-shot image generation. The architecture

combines a pretrained StyleGAN2 backbone, a novel StyleFeatureEditor Inverter and Feature Editor, and SAGE's attribute group editing mechanism. The design aims to enable fine-grained, disentangled, and class-consistent image synthesis from limited input data. A modular overview is as follows:

1. **StyleGAN2 :** Similar to the SAGE framework, the synthesis module of a pretrained StyleGAN2 is used to generate images from the inverted latent space codes output of the GAN inverter (StyleFeatureEditor). Additionally, with the integration of the Feature Editor of SFE, we utilized the feature tensor together with the latent codes to synthesize the resulting image.

2. **StyleFeatureEditor Encoder (SFE)** : SFE by Bobkov et al. (2024) , is a novel method that enables editing in both w-latents and F-latents. This technique not only allows for the reconstruction of finer image details but also ensures their preservation during editing". In the FeatureSAGE framework, SFE is *integrated into* the GAN inversion module $I(\cdot)$ of the original SAGE framework to enhance latent inversion and attribute editing capabilities. According to the observations of Ding et al. (2023), when reconstructing from the experimented inversion methods (pSp , ReStyle, HyperStyle) , there is an observed loss of local details and thus the accuracy of the reconstructed images is lower by over 5% compared to the stadard data. And as noted by Ding et al. (2023) "If we obtain a better inversion method with better reconstruction ability and editability, SAGE will be endowed with a larger potential to further improve downstream classifications with generative augmentation." Thus, with this in mind, we propose to integrate SFE as the GAN Inverter $I(\cdot)$ model for this study together with its Feature Editor $H(\cdot)$ for improved editability and reconstruction.

3.3.1 Framework

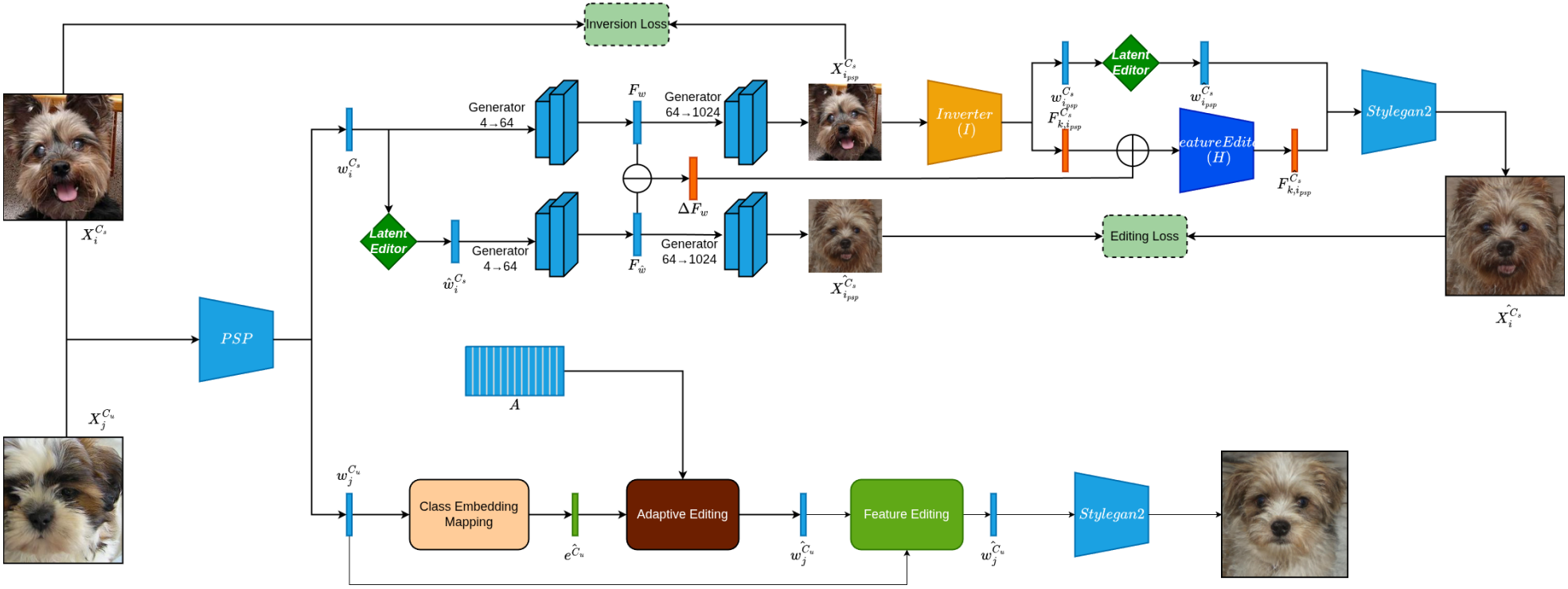

Figure 3.3 FeatureSAGE framework

Figure 3.3 shows the overview of the FeatureSAGE framework.

The training set for seen categories is denoted as D_{train} = $\{x_i^{c_s}\}^S$, which consists of S seen categories. While the testing set for unseen categories is denoted as D_{test} = $\{x_i^{c_u}\}^U$ consists of U unseen categories.

A pretrained pixel2style2pixel encoder PSP takes seen image $X_i^{C_s}$ and predicts $w_i^{C_s} \in W+$. Using the latent editor, which samples random directions from sparse dictionary A, we are able to get an edited image $\hat{w_i^{C_s}}$ which we will use as an editing pair for training the feature editor module. Image $x_{i_psp}^{C_s}$ and features F_w are synthesized from $w_i^{C_s}$, while $\hat{x_{i_psp}^{C_s}}$ and features $\hat{F}_w$ are synthesized from $\hat{w}^{C_s}$ via Stylegan2 generator G. Generated image $x_{i_psp}^{C_s}$ is then passed to a frozen StyleFeatureEditor Inverter Module I which predicts $F_{k,i_psp}^{C_s}$ and $w_{i_psp}^{C_s}$ which is then edited by the latent editor according to sampled direction. Then ΔF_w is calculated, Feature Editor edits $F_{k,i_psp}^{C_s}$ according to ΔF_w.

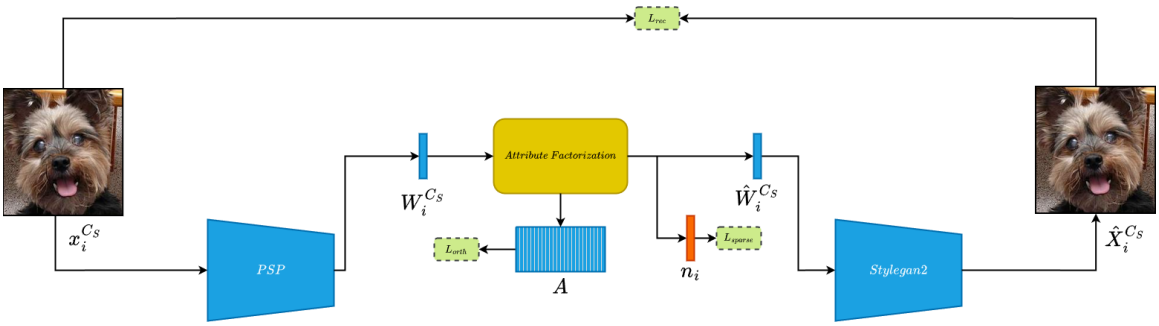

Figure 3.4 *Attribute Factorization Training Framework (Ding et al., 2023)*

To enable disentangled editing, we first need to train a module that learns to identify category irrelevant attributes in the latent space. It is learned through sparse dictionary learning via the difference between the sample embedding and mean latent code of the category.

In the Attribute Factorization block (Fig. 3.5), we retain the same two-step decom-

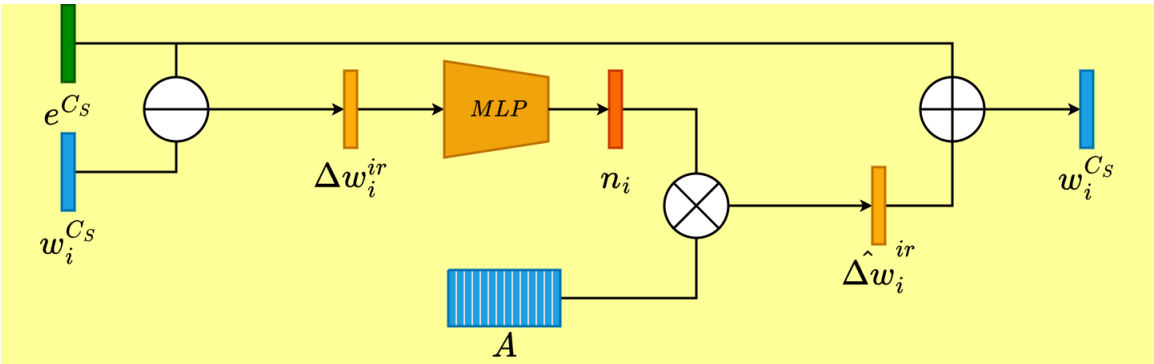

Figure 3.5 *Modified Attribute Factorization Block, Derived from (Ding et al., 2023)*

position as in SAGE—i.e. we compute the residual

$$\Delta w_i^{ir} \;=\; w_i^{C_s} - e^{C_s}$$

and factorize it via the learned dictionary A and MLP-predicted coefficients n_i so that

$$\hat{\Delta w}_i^{ir} \;=\; A\, n_i.$$

This edited residual is then added back to the mean latent to get the edited image $\hat{w_i^{C_s}}$.

Category-irrelevant Attributes are features that influence how an object appears but are not essential to its identity or class membership. Example is that a cat is still a cat even if it faces left or right , the color changes , the background changes , texture, expression, etc. They are attributes that can change the appearance but not the class of the object. According to Ding et al. (2023), we define category-irrelevant editing as $edit_{ir}(\cdot)$ which does not change the category given a latent code w^{C_S} of seen category C_S:

$$edit_{ir}(G(W_i^{C_S})) = G(W_i^{C_S} + \alpha \Delta W^{ir}, H(F_{ki,\Delta W^{ir}})) = \hat{X}_i^{C_S},$$

Category-irrelevant directions are consistent across both known and unknown categories. The same category-irrelevant directions are observed across all known and

unknown categories. Given a latent code $W_i^{C_S}$ and its class embedding e^{C_S}, category-irrelevant direction sample Δw_i^{ir} can be obtained by:

$$\Delta W_i^{ir} = W_i^{C_S} - e^{C_S}.$$

According to Ding et al. (2023) , a global dictionary $A \in \mathbb{R}^{18\times512\times l}$ and a sparse representation $n_i \in \mathbb{R}^{18\times l}$ are learned according to ΔW_i^{ir} using a Sparse Dictionary Learning(SDL) to learn the globally shared category-irrelevant directions. The SDL is as follows:

$$\min_n \|n_i\|_0, \quad s.t. \Delta W_i^{ir} = An_i$$

where A contains all the directions of the category-irrelevant attributes, and $\|\cdot\|_0$ is the L_0 constraint that encourages each element in A to be semantically meaningful.

In practice, according to Ding et al. (2023), it is optimized via an Encoder-Decoder architecture. From ΔW_i^{ir}, we can obtain n_i with a Multi-Layer Perceptron(MLP).

$$n_i = MLP(\Delta W_i^{ir})$$

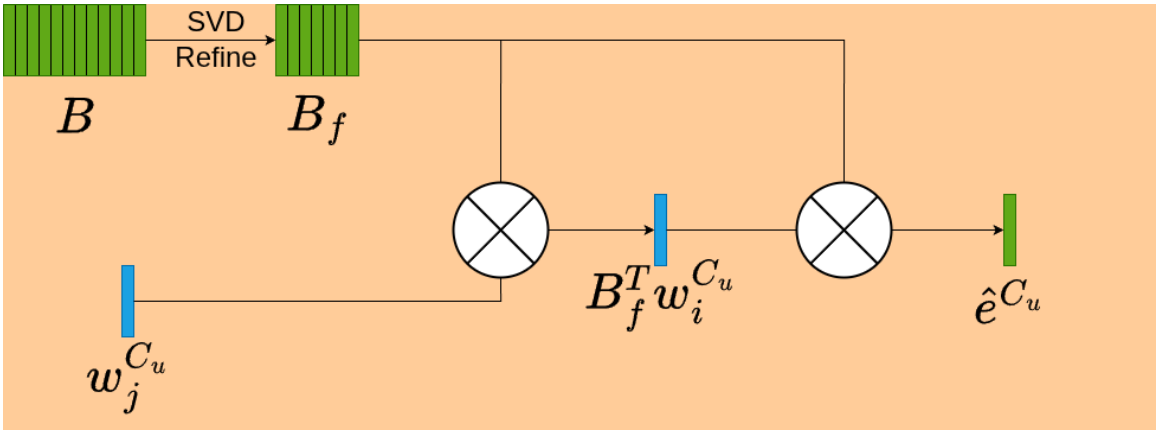

Figure 3.6 *Class Embedding Block , Adapted from (Ding et al., 2023)*

Compared to the Attribute Group Editing (AGE) by Ding et al. (2022), which randomly samples editing directions from A, Stable Attribute Group Editing (SAGE) uses

class embedding mapping. The class-embedding mapping block (Figure 3.6) finds the most important directions in the latent code space for each cohort, identifying which latent directions have the greatest effect on the target attribute group. These "top" directions are then used to weight and steer the subsequent edits. This part of the framework remains unchanged from the SAGE framework.

Specifically , it takes a set of class embedding of S seen categories, B. As stated before , B is the dictionary of category-relevant attributes but since they are based on average of latent codes, it will inevitably contain a small amount of interference from category-irrelevant attributes.

To identify the most important, meaningful directions in B and remove the unwanted noise, we apply singular value decomposition (SVD) (Zhang, 2015). We define the filtered B class embeddings as:

$$B = U_B \sum V_B^*$$

Where B is the matrix of category-relevant class embeddings, U_B is the matrix of left singular vectors of B (an $m \times m$ orthogonal matrix whose columns are the left singular vectors of B) , $\sum$ is a diagonal $m \times n$ matrix containing the singular values of B in descending order (denotes the importance/energy of each direction) and V_B^* is the Conjugate transpose (Hermitian transpose) of the right singular vectors matrix.

According to Ding et al. (2023), the top-t_B directions in U_B are selected as the final category-relevant attribute dictionary $B_f \in \mathbb{R}^{18\times512\times t_B}$.

The class embedding of c_u can now be estimated by the average projection from $W_i^{C_u}$ to B_f using the formula:

$$\hat{e}^{C_u} = \frac{1}{N_u} \sum_{i=1}^{N_u} B_f B_f^T W_i^{C_u},$$

given N_u latent codes of $\{w_i^{C_u}\}_{i=1}^{N_u}$ unseen categories C_u. $\hat{e}^{C_u}$ encodes the category-

relevant attributes and removes the most category-irrelevant attributes.

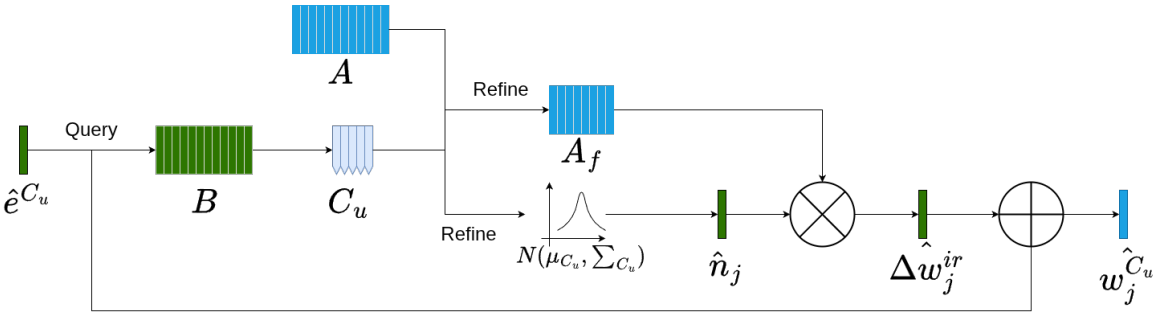

Figure 3.7 *Adaptive Editing Block, from (Ding et al., 2023)*

Figure 3.7 shows the adaptive editing block of FeatureSAGE. Category-irrelevant qualities typically have similar distributions in closely related categories but differ dramatically in more distant ones. Editing photos in a category-aware manner helps to preserve category-specific properties. Therefore we use adaptive editing to achieve this. Given a class e^{C_u} of unseen category C_u, the top-t_C seen categories $C_u = [C_{S_1}, C_{S_2}....C_{S_{t_C}}]$ are first selected according to the distances between their class embeddings (Ding et al., 2023).

To find the attribute-irrelevant editing directions that best fit the unseen category C_u, we first back project ΔW^{ir} into the representation $\hat{n}$:

$$\hat{n} = A^{-1}\Delta W_i^{ir}$$

where A^{-1} is the pseudo-inverse matrix of A. Afterward, we count the mean of the absolute value of $|n|$ across the the similar seen categories C_u (Ding et al., 2023):

$$\overline{|n|}^{C_u} = \frac{1}{t_C}\sum_{t=1}^{t_C}\frac{1}{N_{S_t}}\sum_{i=1}^{N_{S_t}}|\hat{n}_i^{C_{S_t}}|,$$

where $\overline{|n|}^{C_u}$ captures how frequently or strongly certain directions appear across the set C_u, effectively measuring their commonality. For each layer in the W^+ space, we identify the top-t_A directions from the set A by selecting those with the highest values

in $\overline{|n|}^{C_u}$, meaning they are the most consistently present or prominent across C_u. The adaptive category-irrelevant dictionary for C_u is $A_{C_u} \in \mathbb{R}^{18\times512\times t_A}$.

Assuming that the sparse representation n of different categories obeys a Gaussian distribution, we estimate the distribution $\mathcal{N}(\mu_{C_u}, \sum_{C_u})$ by counting the $\hat{n}_i$ of all samples in C_u to automatically generate diverse images. Sampling an arbitrary $\tilde{n}_j$ from $\mathcal{N}(\mu_{C_u}, \sum_{C_u})$ and applying editing to the estimated class embedding, we can generate image $X_j^{C_u}$ by:

$$X_j^{C_u} = G(\hat{e}^{C_u} + \alpha A_{C_u}\tilde{n}_j, H(F_{ki}, \alpha A_{C_u}\tilde{n}_j))$$

where the intensity of manipulation is denoted by α to control diversity in the generated image.

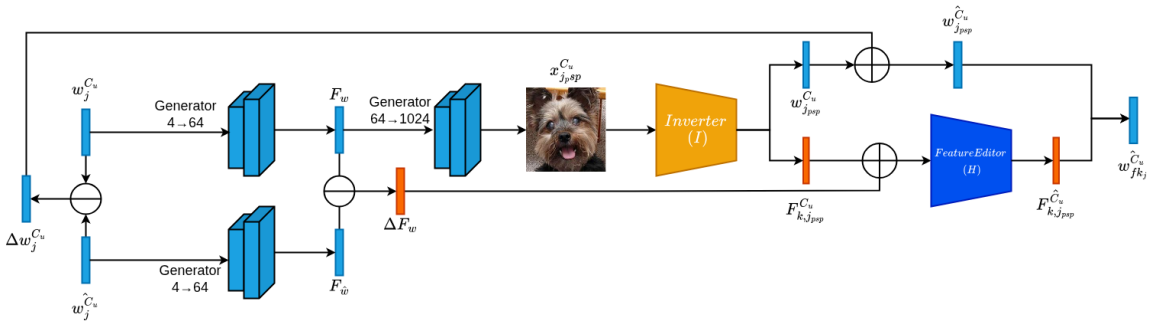

Figure 3.8 *Feature Editing Decoder Block*

Figure 3.8 show the feature editing decoder block. It takes the latent of the original image, and the edited latents after the adaptive editing block as inputs. For a semantic edit on the $W+$ space from the Inverter (I), we first need to calculate the latent difference between the original image latent and the edited latent, denoted by $\Delta w_j^{C_u}$. We also need to get the difference between the feature spaces of both latents at layer 9 for feature injection which is denoted as :

$$\Delta F_w = F_w - F_{\hat{w}}$$

,

Latent $w_{jpsp}^{C_u}$ is then used to generate image while injecting edited features $F_{k,\hat{j_{psp}^{C_u}}}$ back into the 9th layer of the stylegan feature space.

3.3.2 Training Strategy

In the training stage, we first need to train a StlyeGAN2 with images from the seen categories . Similarly, the StyleFeatureEditor Inverter and the PSP encoder are also trained. To minimize resource consumption, the model is trained via a double phase method.

3.3.3 Phase 1: Attribute Factorization Training

Phase 1 involves training the attribute factorization block for disentangled sampling of attribute irrelevant directions. To train the FeatureSAGE framework to effectively generate images in a few-shot environment and ensure generalizability, we adopt a systematic training strategy involving loss optimization, regularization, and evaluation monitoring. Below are the key components of our training pipeline.

The following are the loss functions applied to the attribute factorization block:

- Sparsity Loss (L_{sparse}) : The sparsity loss encourages the learned attribute code to be sparse, so that only a few dictionary directions are active for any image. Concretely, if n_i is the code vector output by an MLP (encoding the differences between an image and the class mean), we approximate an L_0 penalty by using a sigmoid and L_1 norm defined as :

$$L_{sparse} = \|\sigma(\theta_0 n_i - \theta_1)\|_1,$$

  where $\sigma(\cdot)$ denotes he sigmoid function. $\theta_0 \; and \; \theta_1$ are hyper-parameters to control the sparsity. Since the output range from the MLP is very large and unstable, Ding et al. (2023) use Sigmoid activation as a trick to compress the

outputs to [0, 1]. This pushes each entry of n_i toward 0 or 1, so that only a few entries are nonzero. In effect, each image edit uses a sparse combination of attribute directions. By penalizing nonzero entries in the code, L_{sparse} forces each edited image to be produced from only a few directions. This yields disentangled, semantically meaningful edits (each direction corresponds to one attribute)

- Reconstruction Loss (L_{rec}) : The L_2 reconstruction loss ensures that the edited image stays close to the original input. If e^{C_s} is the class-center latent for category c and An_i is the attribute offset, then the generated image is $G(e^{C_s} + An_i)$ (where G is the StyleGAN generator). L_{rec} penalizes the difference between this edited image and the original real image x_i^c.

$$L_{rec} = \|G(e^{C_s} + An_i) - x_i^{C_s}\|_2$$

This formula anchors the editing process: it pulls the output back toward the input image, so that only the intended (irrelevant) attributes change. In other words, it prevents "over-editing." If an edit drastically alters the image, L_{rec} grows large, forcing the network to adjust An_i so the change remains subtle. Overall, it keeps the edited image faithful to the input. This stabilizes training by ensuring that edits do not introduce artifacts or destroy content; it effectively acts like a latent-space autoencoder reconstruction loss.

- Orthogonality Loss (L_{orth}) :The orthogonality loss enforces that the learned "irrelevant" directions do not affect category-relevant features. Let A be the matrix of category-irrelevant directions and let B be a basis (or embedding matrix) for category-relevant attributes. This guarantees that editing along "irrelevant" directions does not leak into the "relevant" subspace. This preserves class identity and semantic consistency: only style or pose may change, not the

object's class. It is defined as :

$$L_{orth} = \|B^T A\|_F^2,$$

where $\|\cdot\|_F^2$ denotes the Frobenius Norm.

The overall loss function for the attribute factorization block is defined as :

$$L_{af} = L_{rec} + \lambda_1 L_{orth} + \lambda_2 L_{sparse},$$

where λ_1 and λ_2 are weights for L_{orth} and L_{sparse} (Ding et al., 2023).

Optimization Algorithm

Training is done using the Adam Optimizer with the same configuration as defined by Ding et al. (2023) , a fixed learning rate of 1×10^{-4} with hyperparameters of $\lambda_1 = 5 \times 10^{-4}$, $\lambda_2 = 5 \times 10^{-3}, \theta_0 = 5 \times 10^{-1}$ and $\theta_1 = -1$.

3.3.4 Phase 2: Feature Editor Training

Phase 2 involves training the Feature Editor module via generating an editing pair using a pretrained psp encoder. Following the method of Bobkov et al. (2024), we generate image $(X, \hat{X})$ where psp takes input image X and predicts w which is then edited by a sampled direction using the adaptive editing method of the original SAGE to generate random image direction within local distribution and sampled sparse A attributes. This ensures that feature editor learns to edit features in a varying directions.

3.4 Evaluation Protocol

3.4.1 Evaluation Metrics

We evaluated the models using quantitative metrics for both reconstruction quality and editing performance, under a few-shot test-time setup.

1. **Fréchet Inception Distance (FID)**

   The Fréchet Inception Distance (FID) is a widely adopted metric for evaluating the quality of images generated by deep generative models, particularly in tasks such as few-shot image generation Heusel et al. (2017); Wang et al. (2023). Rather than comparing images directly at the pixel level, FID operates in a high-level feature space extracted from a pretrained Inception-v3 network Heusel et al. (2017). This makes it effective for capturing both the fidelity (realism) and diversity (variety) of generated samples Borji (2022). The FID score is computed by modeling the feature distributions of real and generated images as multivariate Gaussians Heusel et al. (2017). The FID score is computed by modeling the feature distributions of real and generated images as multivariate Gaussians. The formula for FID is:

$$\text{FID} = \|\mu_r - \mu_g\|_2^2 + \text{Tr}\left(\Sigma_r + \Sigma_g - 2\left(\Sigma_r \Sigma_g\right)^{1/2}\right) \tag{1}$$

   Where:

   - μ_r, Σ_r are the mean and covariance of the Inception features of real images,
   - μ_g, Σ_g are the mean and covariance of the Inception features of generated images,
   - $\|\mu_r - \mu_g\|_2^2$ represents the squared Euclidean distance between the feature means,

- $\mathrm{Tr}(\cdot)$ is the trace operator, which sums the diagonal elements of a matrix,
- $(\Sigma_r \Sigma_g)^{1/2}$ denotes the matrix square root of the product of the covariance matrices.

This formula computes the Fréchet (2-Wasserstein) distance between two multivariate Gaussian distributions—one fitted to real images and the other to generated images in the feature space Heusel et al. (2017). *Lower FID is better.* Since FID measures the distance between real and generated image distributions, a lower FID score indicates that the generated images are more similar to the real images in terms of both quality and diversity. An FID of 0 implies perfect similarity. Therefore, in few-shot image generation, models that achieve lower FID scores are considered to generate more realistic and varied samples Borji (2022); Wang et al. (2023).

2. **Learned Perceptual Image Patch Similarity (LPIPS)**

The Learned Perceptual Image Patch Similarity (LPIPS) is a metric used to evaluate the perceptual similarity between two images. Instead of comparing images at the pixel level, LPIPS compares deep feature activations from a pretrained neural network (such as VGG or AlexNet), which better reflect human perception Zhang et al. (2018a). This makes LPIPS especially useful in few-shot image generation, where we want to evaluate whether generated images are visually close to target images (for fidelity), or measure how diverse the outputs are (for diversity), depending on how it is applied. The LPIPS score between two images x and x' is calculated as:

$$\text{LPIPS}(x, x') = \sum_l \frac{1}{H_l W_l} \sum_{h=1}^{H_l} \sum_{w=1}^{W_l} \left\| w_l \odot \left(\hat{y}^l_{hw}(x) - \hat{y}^l_{hw}(x') \right) \right\|^2$$

where $\hat{y}^l_{hw}(x)$ is the unit-normalized feature vector of image x at layer l and

position (h, w), w_l is a vector of learned weights for each channel at layer l, and H_l, W_l are the height and width of the feature map at layer l Zhang et al. (2018a). When interpreting LPIPS, *lower values are better* in terms of similarity between generated and real images. A lower LPIPS score indicates that the generated image is more perceptually similar to the reference image. On the other hand, a higher LPIPS score may reflect less similarity (in fidelity tasks) or greater diversity (in intra-cluster evaluations of few-shot generation) Zhang et al. (2018a).

3.4.2 Baselines for Comparison

To contextualize the performance of our proposed few-shot image generation model, we compare it against a set of strong, widely accepted baseline methods. These baselines represent different paradigms of generative modeling and are selected to provide a fair and comprehensive comparison.

- AGE (Ding et al., 2022) – Leverages latent space attribute editing on pretrained GANs to produce novel-class images without retraining, by disentangling class-relevant and irrelevant features.
- SAGE (Ding et al., 2023) – Builds on AGE by aggregating support images to create stable latent embeddings and inject class-consistent diversity via attribute mixing.

3.5 Expected Outcome

The study is expected to significantly enhance latent code retrieval from real-world images. By integrating the StyleFeatureEditing module into SAGE framework, results should show stronger reconstruction alongside easier modification without forcing a trade-off. Instead of relying on the initial pSp-based encoder, this updated setup known as FeatureSAGE may handle follow-up jobs more effectively. Tasks like enriching

datasets could benefit greatly, particularly when only a handful of examples exist. Higher-quality inversions are likely to yield closer-to-original visuals while preserving meaning-rich traits needed for edits. Another boost comes from adding the Feature Editor unit, which tends to sharpen output clarity. It helps keep small textures intact and ensures the subject looks consistent even after changes take place.

In order to evaluate FeatureSAGE, performance is checked on two known datasets, VGGFace2 and Flowers102. Image variety and clarity are reviewed through a common tool called Fréchet Inception Distance (FID). Instead of relying only on visuals, perceptual likeness is measured via LPIPS scores that highlight fine differences between pictures. Keeping the person's look intact during edits receives attention using identity similarity assessments. Comparisons unfold alongside recent methods in GAN-based editing and low-data image changes. These side-by-side tests aim to show whether FeatureSAGE holds up more consistently across adjustments. Preserving who someone looks like matters just as much as making realistic tweaks. Overall picture quality enters evaluation not as an afterthought, but woven into each comparison point.

Based on the objectives outlined in this study, the proposed FeatureSAGE framework is expected to achieve the following measurable outcomes:

1. **Lower FID**, reflecting enhanced visual realism and distributional alignment of real and synthetic images.

2. **Higher Learned Perceptual Image Patch Similarity (LPIPS)**, reflecting increased perceptual diversity among edited images.

All of these results are collectively establish the strength of FeatureSAGE as a robust and universal instrument for few-shot image generation, manipulation, and data enhancement in resource-limited learning systems. The research also contribute to the development of the edit-based few-shot approach.

CHAPTER 4 RESULTS AND DISCUSSION

This chapter presents the experimental results of the proposed FeatureSAGE framework and compares its performance with the baseline SAGE model on few-shot image generation tasks. The experiments are carried out on two datasets, FUNIT Animal Faces and 102 Flowers, with a 1-shot image generation task to evaluate the ability of the proposed framework when faced with a limited amount of data. In order to evaluate the performance in high and low resolution settings, the experiments are conducted at 1024x1024 and 256x256 pixels. The quality of the generated images and their perceptual similarity to real images are measured using Fréchet Inception Distance (FID) and Learned Perceptual Image Patch Similarity (LPIPS), respectively. A comparison between the SAGE model with a pSp inverter and the FeatureSAGE framework with StyleFeatureEditor and feature editing is expected to shed light on the effect of improved latent inversion on image quality and editability.

4.1 Quantitative Results

4.1.1 Quantitative Comparison Results between SFSAGE and Baselines

4.2 Analysis of Quantitative Results

This section analyzes the quantitative performance trends observed in Table 4.1, focusing on how the proposed SFSAGE model compares against AGE and SAGE baselines across the Flowers and Animal Faces datasets at both 1024×1024 and 256×256 resolutions using FID and LPIPS metrics.

Resolution	Model	Flowers		Animal Faces	
		FID ↓	LPIPS ↑	FID ↓	LPIPS ↑
1024	AGE (Baseline)	61.57	0.4108	62.13	0.5033
1024	SAGE (Baseline)	54.17	0.4528	60.92	0.4867
1024	**SFSAGE (Ours)**	45.60	0.4209	49.60	0.4811
256	AGE (Baseline)	171.15	0.2108	-	-
256	SAGE (Baseline)	160.12	0.2085	-	-
256	**SFSAGE (Ours)**	145.48	0.1309	-	-

Table 4.1 Merged quantitative comparison of AGE, SAGE, and the proposed SFSAGE across Flowers and Animal Faces datasets at 256 and 1024 resolutions. Lower FID and higher LPIPS values indicate better performance. The 256×256 Animal Faces results were not evaluated.

4.2.1 102 Flowers

102 Flowers at 1024×1024

On a high resolution setting (1024×1024), SFSAGE obtains an FID of 45.60, which corresponds to a 15.8% improvement over SAGE (54.17) and a 25.9% improvement over AGE (61.57). This substantial FID reduction demonstrates improved distributional alignment with the real flower images in the Inception feature space. In terms of perceptual diversity, SFSAGE achieves an LPIPS of 0.4209, compared to SAGE (0.4528) and AGE (0.4108). This represents a 7.0% decrease relative to SAGE and a 2.5% increase relative to AGE. The LPIPS values across all three methods range from 0.4108 to 0.4528, indicating that perceptual diversity remains relatively consistent across the approaches. The results show that SFSAGE achieves FID improvements of 15.8-25.9% while maintaining LPIPS within 7.0% of the baseline methods. This pattern differs from the 256×256 resolution results on the same dataset, where SFSAGE achieved FID improvements of 9.1-15.0% but with LPIPS reductions of 37.2-37.9%.

102 Flowers at 256×256

At a lower resolution of 256×256, SFSAGE achieves an FID of 145.48, which outperforms SAGE (160.12) by 9.1% and AGE (171.15) by 15.0%. This demonstrates continued FID

improvements across resolutions, though the percentage gains are smaller compared to the 1024×1024 setting (15.8% and 25.9% improvements). In terms of perceptual diversity, SFSAGE achieves an LPIPS of 0.1309, which is 37.2% lower than SAGE (0.2085) and 37.9% lower than AGE (0.2108). The LPIPS values across methods range from 0.1309 to 0.2108 at this resolution, compared to 0.4108 to 0.4528 at 1024×1024, indicating overall lower perceptual diversity at the reduced resolution. Comparing across resolutions, SFSAGE shows different performance patterns: at 1024×1024, FID improvements of 15.8-25.9% are accompanied by LPIPS changes within ±7.0%, while at 256×256, FID improvements of 9.1-15.0% are accompanied by LPIPS reductions of 37.2-37.9%. The LPIPS value decreases from 0.4209 to 0.1309 when resolution is reduced from 1024×1024 to 256×256.

4.2.2 Animal Faces

Animal Faces HQ at 1024×1024

On the Animal Faces HQ dataset at 1024×1024, SFSAGE achieved an FID of 49.60, outperforming SAGE (60.92) by 18.6% and AGE (62.13) by 20.2%. This reduction in FID demonstrates SFSAGE's superior distributional alignment with the real data distribution in the Inception feature space. In terms of perceptual diversity, SFSAGE achieves an LPIPS of 0.4811, compared to SAGE (0.4867) and AGE (0.5033). This represents a 1.2% decrease relative to SAGE and a 4.4% decrease relative to AGE. The LPIPS differences across all three methods are relatively small, with values ranging from 0.4811 to 0.5033, indicating comparable levels of perceptual diversity in the VGG feature space. The results show that SFSAGE achieves substantial FID improvements (18.6-20.2%) while maintaining LPIPS values within 4.4% of the baseline methods. This contrasts with the Flowers dataset at 256×256 resolution, where SFSAGE showed larger LPIPS reductions (37.2-37.9%) alongside FID improvements of 9.1-15.0%.

4.3 Qualitative Evaluation

4.3.1 Visual Comparison Across Models

4.3.2 Visual Observations

4.4 Failure Cases and Limitations

During experimentation and evaluation of the FeatureSAGE framework, several limitations and failure cases were observed:

1. Built using a pretrained StyleGAN2 generator, FeatureSAGE requires fine-tuning on specific data to enable accurate image reconstruction and generation. Because of this setup, its use remains limited to areas where such pre-trained models are already available. When the base model lacks diversity - say, missing certain categories - those gaps carry forward into results. Oddly enough, flaws embedded in the original network's hidden representations often surface during editing tasks. Depending on training history, these quirks may distort outcomes or skew how traits appear across faces. As a result, changes made through the system can reflect old imbalances instead of producing balanced updates.

2. Computation Overhead. Adding the StyleFeatureEditor inverter along with the Feature Editor demands more processing than the base SAGE setup. These parts boost reconstruction quality yet require greater resources. Performance gains come at a cost in speed. Though results improve, the system now takes longer per operation. Efficiency drops slightly due to added layers. Each forward pass grows heavier. More complex steps slow down inference. Trade-offs appear in timing versus output fidelity. Higher precision arrives with increased load. Processing time rises alongside capability. System throughput narrows somewhat. Resource demand climbs without drastic changes elsewhere.

Due to higher demands on processing resources, these features take up more

GPU memory while slowing down inference. As a result, performance can drop when handling big datasets or time-sensitive tasks. Focusing on different scales of attributes could open new paths forward. Still, adjusting how strongly edits are applied matters just as much. One path involves testing models other than StyleGAN2, which might ease reliance on a single framework. Generalization may get stronger when diverse designs enter the picture. Exploring such options becomes essential given current constraints.

CHAPTER 5 SUMMARY

5.1 Summary of Findings

5.1.1 Summary of Contributions

1. Integration of StyleFeatureEditor (SFE) Encoder FeatureSAGE integrates StyleFeatureEditor Encoder on the original SAGE framework, enabling improved latent representations that better capture distributional characteristics of the data across resolutions.

2. Integration of Feature Editor The Feature Editor module allows fine-grained and semantically consistent modifications by operating directly on intermediate feature representations, enhancing the expressiveness of attribute edits.

3. Resolution-Dependent and Dataset-Adaptive Editing FeatureSAGE adapts its editing strategies according to both image resolution and dataset characteristics, ensuring appropriate trade-offs between edit diversity, stability, and realism.

4. Comprehensive Benchmarking Against Baselines The framework demonstrates robust performance across structurally diverse datasets (Flowers and Animal Faces) and multiple resolutions, providing a generalizable approach to few-shot image generation while balancing perceptual expressiveness and identity preservation.

5.1.2 Empirical Outcomes

Quantitative Performance FeatureSAGE consistently improves distributional realism across datasets and resolutions. Key metrics include:

- **FID improvements:** At 1024×1024, FeatureSAGE reduces FID by 25.9% on Flowers

(45.60 vs. 61.57 for AGE) and 15.8% (vs. 54.17 for SAGE), and by 20.2% on Animal Faces (49.60 vs. 62.13 for AGE) and 18.6% (vs. 60.92 for SAGE). At 256×256 on Flowers, reductions are 15.0% (vs. AGE) and 9.1% (vs. SAGE).

- **LPIPS performance:** On Flowers at 1024×1024, FeatureSAGE achieves an LPIPS of 0.4209, which is 7.0% lower than SAGE (0.4528) and 2.5% higher than AGE (0.4108), maintaining competitive perceptual diversity. At 256×256, LPIPS decreases to 0.1309, representing a 37.2% reduction compared to SAGE (0.2085) and 37.9% compared to AGE (0.2108). On Animal Faces at 1024×1024, LPIPS is 0.4811, within 1.2% of SAGE (0.4867) and 4.4% of AGE (0.5033).

Dataset and Resolution-Adaptive Trends The framework demonstrates resolution-dependent behavior:

- At 1024×1024 resolution, FeatureSAGE achieves substantial FID improvements (15.8-25.9%) while maintaining LPIPS within 7.0% of baseline methods across both datasets.

- At 256×256 resolution on Flowers, FID improvements are smaller (9.1-15.0%) but accompanied by larger LPIPS reductions (37.2-37.9%), indicating reduced perceptual diversity at lower resolutions.

- LPIPS values decrease substantially from high to low resolution on Flowers (0.4209 to 0.1309), while the pattern differs across datasets based on their structural characteristics.

5.2 Limitations

Though FeatureSAGE holds promise in attribute group editing and limited example image synthesis, some constraints observed during testing indicate that there is scope for further research. The first is the reliance on a pre-trained StyleGAN2 generator,

which restricts the application of this tool only to domains where such generators have been developed, and any biases in the model may have been passed on to the output. The second is the potential performance impact of integrating the StyleFeatureEditor with the Feature Editor, which increases computational requirements. Future research could focus on other base models, incorporate a multi-level feature approach to improve consistency between edits, optimize computation for more general or faster models, and extend few-shot adaptation to work well even on entirely new classes.

5.3 future Work

5.3.1 Efficiency Optimizations

5.3.2 Broader Applications

5.4 Conclusion

BIBLIOGRAPHY

Rameen Abdal, Peihao Zhu, Niloy J. Mitra, and Peter Wonka. Styleflow: Attribute-conditioned exploration of stylegan-generated images using conditional continuous normalizing flows. *ACM Transactions on Graphics*, 40(3):1–21, May 2021. ISSN 1557-7368. doi: 10.1145/3447648. URL http://dx.doi.org/10.1145/3447648.

Yuval Alaluf, Or Patashnik, and Daniel Cohen-Or. Restyle: A residual-based stylegan encoder via iterative refinement, 2021. URL https://arxiv.org/abs/2104.02699.

Amit H. Bermano, Rinon Gal, Yuval Alaluf, Ron Mokady, Yotam Nitzan, Omer Tov, Or Patashnik, and Daniel Cohen-Or. State-of-the-art in the architecture, methods and applications of stylegan, 2022. URL https://arxiv.org/abs/2202.14020.

Denis Bobkov, Vadim Titov, Aibek Alanov, and Dmitry Vetrov. The devil is in the details: Stylefeatureeditor for detail-rich stylegan inversion and high quality image editing, 2024. URL https://arxiv.org/abs/2406.10601.

Ali Borji. Pros and cons of gan evaluation measures. *Computer Vision and Image Understanding*, 215:103329, 2022.

Yu Cao and Shaogang Gong. Few-shot image generation by conditional relaxing diffusion inversion, 2024. URL https://arxiv.org/abs/2407.07249.

Xi Chen, Yan Duan, Rein Houthooft, John Schulman, Ilya Sutskever, and Pieter Abbeel. Infogan: Interpretable representation learning by information maximizing generative adversarial nets, 2016. URL https://arxiv.org/abs/1606.03657.

Arkabandhu Chowdhury, Mingchao Jiang, Swarat Chaudhuri, and Chris Jermaine. Few-

shot image classification: Just use a library of pre-trained feature extractors and a simple classifier, 2021. URL https://arxiv.org/abs/2101.00562.

Louis Clouâtre and Marc Demers. Figr: Few-shot image generation with reptile, 2019. URL https://arxiv.org/abs/1901.02199.

Gilad Cohen and Raja Giryes. Generative adversarial networks, 2022. URL https://arxiv.org/abs/2203.00667.

Antonia Creswell, Tom White, Vincent Dumoulin, Kai Arulkumaran, Biswa Sengupta, and Anil A. Bharath. Generative adversarial networks: An overview. *IEEE Signal Processing Magazine*, 35(1):53–65, January 2018. ISSN 1558-0792. doi: 10.1109/msp.2017.2765202. URL http://dx.doi.org/10.1109/MSP.2017.2765202.

Guanqi Ding, Xinzhe Han, Shuhui Wang, Shuzhe Wu, Xin Jin, Dandan Tu, and Qingming Huang. Attribute group editing for reliable few-shot image generation, 2022. URL https://arxiv.org/abs/2203.08422.

Guanqi Ding, Xinzhe Han, Shuhui Wang, Xin Jin, Dandan Tu, and Qingming Huang. Stable attribute group editing for reliable few-shot image generation, 2023. URL https://arxiv.org/abs/2302.00179.

Ian Goodfellow. Nips 2016 tutorial: Generative adversarial networks, 2017. URL https://arxiv.org/abs/1701.00160.

Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks, 2014. URL https://arxiv.org/abs/1406.2661.

Parul Gupta, Munawar Hayat, Abhinav Dhall, and Thanh-Toan Do. Conditional distribution modelling for few-shot image synthesis with diffusion models, 2024. URL https://arxiv.org/abs/2404.16556.

Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In *Advances in Neural Information Processing Systems*, 2017.

Yan Hong, Li Niu, Jianfu Zhang, Jing Liang, and Liqing Zhang. Deltagan: Towards diverse few-shot image generation with sample-specific delta, 2022. URL https://arxiv.org/abs/2009.08753.

Erik Härkönen, Aaron Hertzmann, Jaakko Lehtinen, and Sylvain Paris. Ganspace: Discovering interpretable gan controls, 2020. URL https://arxiv.org/abs/2004.02546.

Tero Karras, Timo Aila, Samuli Laine, and Jaakko Lehtinen. Progressive growing of gans for improved quality, stability, and variation, 2018. URL https://arxiv.org/abs/1710.10196.

Tero Karras, Samuli Laine, and Timo Aila. A style-based generator architecture for generative adversarial networks, 2019. URL https://arxiv.org/abs/1812.04948.

Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of stylegan, 2020. URL https://arxiv.org/abs/1912.04958.

Jianhui Li, Jianmin Li, Haoji Zhang, Shilong Liu, Zhengyi Wang, Zihao Xiao, Kaiwen Zheng, and Jun Zhu. Preim3d: 3d consistent precise image attribute editing from a single image, 2023. URL https://arxiv.org/abs/2304.10263.

Hongyu Liu, Yibing Song, and Qifeng Chen. Delving stylegan inversion for image editing: A foundation latent space viewpoint, 2023. URL https://arxiv.org/abs/2211.11448.

Ming-Yu Liu, Xun Huang, Arun Mallya, Tero Karras, Timo Aila, Jaakko Lehtinen, and Jan Kautz. Few-shot unsueprvised image-to-image translation. In *arxiv*, 2019a.

Ming-Yu Liu, Xun Huang, Arun Mallya, Tero Karras, Timo Aila, Jaakko Lehtinen, and Jan Kautz. Few-shot unsupervised image-to-image translation, 2019b. URL https://arxiv.org/abs/1905.01723.

Andrew Melnik, Maksim Miasayedzenkau, Dzianis Makaravets, Dzianis Pirshtuk, Eren Akbulut, Dennis Holzmann, Tarek Renusch, Gustav Reichert, and Helge Ritter. Face generation and editing with stylegan: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(5):3557–3576, May 2024. ISSN 1939-3539. doi: 10.1109/tpami.2024.3350004. URL http://dx.doi.org/10.1109/TPAMI.2024.3350004.

Sangwoo Mo, Minsu Cho, and Jinwoo Shin. Freeze the discriminator: a simple baseline for fine-tuning gans, 2020. URL https://arxiv.org/abs/2002.10964.

Maria-Elena Nilsback and Andrew Zisserman. Automated flower classification over a large number of classes. In *Indian Conference on Computer Vision, Graphics and Image Processing*, Dec 2008.

Utkarsh Ojha, Yijun Li, Jingwan Lu, Alexei A. Efros, Yong Jae Lee, Eli Shechtman, and Richard Zhang. Few-shot image generation via cross-domain correspondence, 2021. URL https://arxiv.org/abs/2104.06820.

Or Patashnik, Zongze Wu, Eli Shechtman, Daniel Cohen-Or, and Dani Lischinski. Styleclip: Text-driven manipulation of stylegan imagery, 2021. URL https://arxiv.org/abs/2103.17249.

Elad Richardson, Yuval Alaluf, Or Patashnik, Yotam Nitzan, Yaniv Azar, Stav Shapiro, and Daniel Cohen-Or. Encoding in style: a stylegan encoder for image-to-image translation, 2021. URL https://arxiv.org/abs/2008.00951.

Pegah Salehi, Abdolah Chalechale, and Maryam Taghizadeh. Generative adversarial networks (gans): An overview of theoretical model, evaluation metrics, and recent developments, 2020. URL https://arxiv.org/abs/2005.13178.

Yujun Shen and Bolei Zhou. Closed-form factorization of latent semantics in gans, 2021. URL https://arxiv.org/abs/2007.06600.

Yujun Shen, Jinjin Gu, Xiaoou Tang, and Bolei Zhou. Interpreting the latent space of gans for semantic face editing, 2020. URL https://arxiv.org/abs/1907.10786.

Yisheng Song, Ting Wang, Subrota K Mondal, and Jyoti Prakash Sahoo. A comprehensive survey of few-shot learning: Evolution, applications, challenges, and opportunities, 2022. URL https://arxiv.org/abs/2205.06743.

Omer Tov, Yuval Alaluf, Yotam Nitzan, Or Patashnik, and Daniel Cohen-Or. Designing an encoder for stylegan image manipulation, 2021. URL https://arxiv.org/abs/2102.02766.

Andrey Voynov and Artem Babenko. Unsupervised discovery of interpretable directions in the gan latent space, 2020. URL https://arxiv.org/abs/2002.03754.

Yikai Wang, Chen Li, and Chen Change Loy. Attribute group editing for reliable few-shot image generation. *arXiv preprint arXiv:2203.08422*, 2023.

Zongze Wu, Dani Lischinski, and Eli Shechtman. Stylespace analysis: Disentangled controls for stylegan image generation, 2020. URL https://arxiv.org/abs/2011.12799.

Weihao Xia, Yulun Zhang, Yujiu Yang, Jing-Hao Xue, Bolei Zhou, and Ming-Hsuan Yang. Gan inversion: A survey, 2022. URL https://arxiv.org/abs/2101.05278.

Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings*

of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pages 586–595, 2018a.

Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric, 2018b. URL https://arxiv.org/abs/1801.03924.

Ruicheng Zhang, Guoheng Huang, Yejing Huo, Xiaochen Yuan, Zhizhen Zhou, Xuhang Chen, and Guo Zhong. Tage: Trustworthy attribute group editing for stable few-shot image generation, 2024. URL https://arxiv.org/abs/2410.17855.

Zhihua Zhang. The singular value decomposition, applications and beyond, 2015. URL https://arxiv.org/abs/1510.08532.

Yunqing Zhao, Henghui Ding, Houjing Huang, and Ngai-Man Cheung. A closer look at few-shot image generation, 2023. URL https://arxiv.org/abs/2205.03805.